



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

FlixFinderTV

Descubrimiento y recomendaciones
personalizadas de contenido audiovisual



Presentado por Iván Estépar Rebollo
en Universidad de Burgos
10 de junio de 2025
Tutora: Ana Serrano Mamolar

Resumen

El entretenimiento mediante películas y series se ha convertido en una de las principales formas de ocio actual. Sin embargo, la gran cantidad de contenido disponible hace que sea cada vez más complicado para los usuarios encontrar recomendaciones verdaderamente alineadas con sus preferencias. La variedad abrumadora, junto con la diversidad de géneros y formatos, puede generar confusión y saturación, dificultando que cada usuario acceda fácilmente a contenido que realmente le interese y disfrute. Este proyecto propone una aplicación móvil *full stack* diseñada para facilitar la exploración y recomendación de contenido audiovisual adaptado a las preferencias individuales.

La aplicación FlixFinderTV ofrece una experiencia intuitiva, con sugerencias basadas en los gustos del usuario y opciones para explorar por género, estrenos y títulos populares. También incluye funciones como listas de favoritos, moderación de comentarios y acceso a datos sin conexión, con el objetivo de mejorar la experiencia de búsqueda y visualización.

El propósito es ofrecer una herramienta práctica y accesible que ayude a los usuarios a descubrir películas y series de forma eficiente y personalizada, sin complicaciones.

Descriptores

Recomendación personalizada, películas, series, aplicación móvil, gestión de favoritos, listas personalizadas, interacción social, comentarios y valoraciones, comunidad de usuarios, entretenimiento digital, desarrollo de aplicaciones, Kotlin, Jetpack Compose, Firebase, Room.

Abstract

Entertainment through movies and series has become one of the main forms of leisure today. However, the vast amount of available content makes it increasingly difficult for users to find recommendations truly aligned with their preferences. The overwhelming variety, along with the diversity of genres and formats, can create confusion and saturation, making it hard for each user to easily access content they genuinely find interesting and enjoyable. This project proposes a full-stack mobile application designed to facilitate the exploration and recommendation of audiovisual content tailored to individual preferences.

The FlixFinderTV app offers an intuitive experience, with suggestions based on user tastes and options to explore by genre, new releases, and popular titles. It also includes features such as favorites lists, comment moderation, and offline data access, aiming to enhance the search and viewing experience.

The goal is to provide a practical and accessible tool that helps users efficiently and personally discover movies and series without complications.

Keywords

Personalized recommendations, movies, TV shows, mobile app, offline, favorites management, personalized lists, social interaction, comments and ratings, user community, digital entertainment, application development, cinema, audiovisual content, Kotlin, Jetpack Compose, Firebase, Room.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
2. Objetivos del proyecto	5
2.1. Objetivos funcionales	5
2.2. Objetivos técnicos	6
2.3. Objetivos personales	8
3. Conceptos teóricos	11
3.1. Arquitectura MVVM (<i>Model-View-ViewModel</i>)	11
3.2. API REST	11
3.3. Corrutinas en <i>Kotlin</i>	12
3.4. Persistencia Local	13
3.5. Base de Datos No Relacional	14
3.6. Gestión Reactiva del Estado: <i>LiveData</i> y <i>StateFlow</i>	15
4. Técnicas y herramientas	17
4.1. Metodología de Desarrollo	17
4.2. Entorno de Desarrollo	17
4.3. Técnicas y Arquitectura	19
4.4. Diseño de Interfaz de Usuario	20
4.5. Obtención de Datos	21

4.6. Almacenamiento de Datos	22
4.7. Inteligencia Artificial	24
4.8. Desarrollo de la Memoria y Anexos	24
4.9. Prototipado del Esquema de Navegación	25
5. Aspectos relevantes del desarrollo del proyecto	31
5.1. Metodologías Aplicadas	31
5.2. Inicio del Proyecto	32
5.3. <i>Frontend</i>	33
5.4. <i>Backend</i>	35
5.5. Modo <i>Offline</i>	39
5.6. Despliegue	40
5.7. Testing	42
5.8. Métricas de calidad	44
5.9. Créditos a librerías y contribuciones externas	44
6. Trabajos relacionados	47
6.1. Rotten Tomatoes	47
6.2. JustWatch	48
6.3. FilmAffinity	49
6.4. Trakt.tv	50
6.5. Letterboxd (Versión Simplificada)	51
6.6. IMDb	52
7. Conclusiones y Líneas de trabajo futuras	53
7.1. Conclusiones	53
7.2. Líneas de trabajo futuras	55
Bibliografía	57

Índice de figuras

3.1. Esquema del funcionamiento de una API REST. Fuente: aprendiendoarduino.wordpress.com	12
3.2. Esquema del funcionamiento de las corrutinas. Fuente: developer.android.com	13
4.1. Esquema de la estructura MVVM	20
4.2. Método de acceso para los usuarios	23
4.3. Prototipo inicial de la aplicación	27
4.4. Esquema de navegación de la aplicación	28
4.5. Análisis de calidad del código en <i>SonarCloud</i>	29
5.1. Logo de la aplicación	32
5.2. Organización de carpetas en el frontend	33
5.3. Organización de carpetas en <i>Firestore</i>	35
5.4. Organización de carpetas en <i>Room</i>	37
5.5. Función que comprueba la conexión	39
5.6. Aplicaciones guardadas en <i>Appetize</i>	40
5.7. Emulador de <i>Appetize</i>	41
5.8. Organización de carpetas en el <i>testing</i>	42
5.9. Resultados de los <i>tests</i>	43
6.1. Logo de Rotten Tomatoes	47
6.2. Logo de JustWatch	48
6.3. Logo de FilmAffinity	49
6.4. Logo de Trakt.tv	50
6.5. Logo de Letterboxd	51
6.6. Logo de IMDb	52

Índice de tablas

4.1. Herramientas y tecnologías utilizadas en cada parte del proyecto	30
---	----

1. Introducción

“El cine es un espejo del mundo, pero también una ventana hacia otro.”

— **Jean-Luc Godard** [32]

Hoy en día, con la gran cantidad de opciones de películas y series en las plataformas de *streaming*, elegir qué ver puede ser todo un reto. La abundancia de contenido puede resultar abrumadora y, en muchas ocasiones, terminamos pasando más tiempo buscando algo que realmente nos interese que disfrutando de lo que realmente queremos ver. FlixFinderTV surge como solución a este problema, ofreciendo una experiencia completamente personalizada que hace mucho más fácil encontrar el contenido que se ajusta a nuestros gustos.

Lo que hace especial a FlixFinderTV es que se adapta a tus géneros favoritos. Desde el primer uso, eliges qué tipo de contenido te gusta, y la app te muestra películas y series que encajan con tus preferencias. Así, no pierdes tiempo buscando: recibes recomendaciones directas según lo que más disfrutas.

Pero FlixFinderTV no se queda solo en la personalización. También nos da la oportunidad de interactuar con otros usuarios, compartiendo opiniones y consultando las reseñas de otros usuarios. Esta característica social hace que la experiencia sea aún más rica, ayudándonos a descubrir títulos que de otro modo podríamos haber pasado por alto.

Esta aplicación se desarrolla con el propósito de resolver el problema de la saturación de opciones en plataformas de *streaming*, ofreciendo una forma práctica, rápida y entretenida de descubrir contenido audiovisual adaptado a los intereses personales de cada usuario.

Estructura de la Documentación

Este proyecto está compuesto por dos documentos principales: la memoria y los anexos. La memoria tiene como propósito principal documentar de manera detallada todo el proceso realizado durante el desarrollo del proyecto. Por otro lado, los anexos proporcionan información adicional que complementa la memoria y facilita la comprensión del trabajo realizado.

La estructura de la memoria es la siguiente:

- **Introducción:** Esta sección proporciona una visión general del proyecto y se centra en el propósito de la aplicación, así como el problema que busca resolver.
- **Objetivos:** Aquí se detallan los objetivos generales y específicos del proyecto. Se detallan las metas propuestas y cómo se abordaron para alcanzarlas.
- **Conceptos Teóricos:** En esta parte, se explican los conceptos y fundamentos teóricos clave que sustentan el proyecto. Se abordan las bases científicas y técnicas necesarias para comprender las decisiones tomadas durante el desarrollo.
- **Técnicas y Herramientas:** En esta sección se describen las metodologías, tecnologías y herramientas utilizadas a lo largo del proyecto. Se incluyen detalles sobre lenguajes de programación, *frameworks*, librerías y otros recursos técnicos esenciales para su desarrollo.
- **Aspectos Relevantes del Desarrollo:** Aquí se destacan los aspectos más importantes del proceso de desarrollo, incluyendo los desafíos enfrentados y las decisiones clave que marcaron el rumbo del proyecto.
- **Trabajos Relacionados:** En esta sección, se realiza un análisis de trabajos previos y proyectos similares. Se revisan las investigaciones o desarrollos previos que aborden problemas semejantes, comparando sus enfoques con el adoptado en este proyecto.
- **Conclusiones y Líneas de Trabajo Futuras:** En esta parte se reflexiona sobre los logros alcanzados, se resumen los resultados obtenidos y se proponen posibles mejoras o nuevas líneas de trabajo que podrían continuar el desarrollo del proyecto.

En cuanto a los anexos, la estructura es la siguiente:

- **Plan de Proyecto:** En este apartado se detalla cómo se organizó el trabajo a lo largo del tiempo. Se explican las etapas del desarrollo, los plazos que se marcaron y los recursos necesarios para sacar adelante el proyecto.
- **Requisitos:** Aquí se recogen las funcionalidades que debe cumplir la aplicación y otros aspectos importantes que aseguran su correcto funcionamiento. También se incluyen los casos de uso, que muestran de forma sencilla cómo interactúa el usuario con el sistema.
- **Diseño:** En esta sección se describe la arquitectura interna del sistema. Se presentan un diagrama arquitectónico, un esquema de la base de datos y otros recursos visuales que facilitan la comprensión de la estructura técnica del proyecto.
- **Manual del Programador:** Pensado para otros desarrolladores, este manual explica cómo está organizado el código, qué convenciones se han seguido y cómo se puede instalar y configurar el proyecto para trabajar con él sin problemas.
- **Manual del Usuario:** En este documento se explica de forma clara y práctica cómo utilizar la aplicación. Se incluyen pasos detallados, ejemplos y capturas de pantalla para que cualquier persona pueda manejarla sin dificultad.
- **Sostenibilización Curricular:** Este anexo muestra cómo se han aplicado, durante el desarrollo del TFG, conocimientos técnicos y éticos adquiridos en el grado, integrando principios de sostenibilidad, accesibilidad y protección de datos.

Materiales Adjuntos

Los materiales adjuntos a estos documentos son los siguientes:

- El **código fuente al completo del proyecto**, que incluye todos los archivos necesarios para compilar el proyecto, así como las dependencias requeridas para su funcionamiento.
- El archivo **APK compilado de la aplicación**, listo para ser instalado en dispositivos *Android*. Este formato es el estándar de distribución e instalación de aplicaciones para esta plataforma, permitiendo empaquetar todos los componentes de una *app* en un solo archivo para su ejecución.

2. Objetivos del proyecto

2.1. Objetivos funcionales

Los objetivos funcionales son aquellos que definen las características y funcionalidades previstas al inicio del proyecto para su implementación en la aplicación. Estas funcionalidades están orientadas a ofrecer una experiencia enriquecedora para el usuario y abarcan desde la recomendación de contenido hasta la incorporación de una dimensión social en la aplicación. Entre los objetivos funcionales se incluyen los siguientes:

- **Ofrecer recomendaciones personalizadas de películas y series:** La aplicación debe generar sugerencias adaptadas a los intereses del usuario, analizando tanto sus gustos como las valoraciones que ha otorgado, para recomendar contenido que se ajuste a sus preferencias y hábitos de visualización.
- **Permitir la búsqueda y filtrado de contenido:** Los usuarios deben poder buscar y filtrar películas y series por título, género y tipo (película o serie). Además, debe ser posible ordenar los resultados por popularidad.
- **Proporcionar información detallada sobre cada título:** La aplicación debe ofrecer detalles completos sobre cada película o serie, incluyendo sinopsis, calificaciones, género, fecha de estreno, duración, director y demás información relevante para que el usuario pueda tomar decisiones informadas sobre qué ver.
- **Incluir una sección de comentarios:** Los usuarios deben poder dejar comentarios y opiniones sobre los títulos que han visto, fomentando la

interacción y el intercambio de experiencias entre los usuarios. Esta sección debe ser moderada para evitar contenido inapropiado.

- **Permitir la creación de listas de favoritos:** Mi aplicación debe ofrecer a los usuarios la posibilidad de agregar películas y series a una lista de favoritos para que puedan acceder fácilmente a su contenido preferido en el futuro.
- **Incluir un modo trivia:** Se debe incluir un modo de juego interactivo en el que los usuarios puedan poner a prueba sus conocimientos sobre películas y series mediante preguntas con opciones de respuesta. Este modo debe tener una interfaz atractiva y divertida.
- **Garantizar una interfaz intuitiva y fácil de usar:** Mi aplicación debe ser intuitiva, con un diseño atractivo y sencillo de navegar, que permita al usuario encontrar fácilmente lo que busca sin complicaciones.
- **Ofrecer funcionalidad *offline*:** Mi aplicación debe permitir a los usuarios acceder a contenido previamente cargado sin necesidad de conexión a Internet. Esto es especialmente útil para usuarios con acceso limitado a datos móviles o conexiones de baja calidad.
- **Permitir el descubrimiento de géneros:** Se debe incluir una sección que permita explorar películas y series agrupadas por género, facilitando el descubrimiento de nuevo contenido relacionado con sus intereses.
- **Mostrar secciones de nuevos estrenos y próximos lanzamientos:** Mi aplicación debe mantener a los usuarios informados sobre los estrenos más recientes y las próximas películas y series que estarán disponibles para su visualización.

2.2. Objetivos técnicos

Los objetivos técnicos se refieren a los aspectos más específicos del desarrollo de la aplicación, incluyendo las decisiones tecnológicas y las soluciones adoptadas para resolver retos o desafíos técnicos. Algunos de estos objetivos son:

- **Implementar la arquitectura MVVM:** Se utilizará la arquitectura *Model-View-ViewModel* (MVVM) para garantizar que el código de la aplicación sea modular, fácilmente mantenible y escalable, separando claramente la lógica de la interfaz de usuario de la lógica de negocio.

- **Utilizar un servicio externo para la actualización de la base de datos:** La aplicación debe estar integrada con un servicio externo que permita mantener actualizada la base de datos de películas y series, asegurando que los usuarios siempre tengan acceso a información relevante y reciente.
- **Diseñar la aplicación de manera que pueda soportar futuras mejoras y ampliaciones:** La estructura de la aplicación debe ser flexible y extensible para que pueda adaptarse fácilmente a nuevas funcionalidades y cambios tecnológicos en el futuro.
- **Implementar buenas prácticas en el desarrollo:** Seguir las mejores prácticas de desarrollo de software, como la gestión adecuada de dependencias, la realización de pruebas unitarias y la integración de herramientas de control de calidad de código como *SonarCloud*.
- **Integrar una solución en la nube para autenticación y gestión de datos en tiempo real con una base de datos remota:** Utilizar un sistema de autenticación seguro para la gestión de usuarios y una base de datos remota para la sincronización y almacenamiento eficiente de los datos de la aplicación, mejorando la experiencia del usuario y facilitando la gestión remota del contenido.
- **Integrar una base de datos local con *Room*:** Utilizar *Room* como base de datos local para almacenar el contenido más relevante, permitiendo el acceso *offline* a las películas y series sin necesidad de conexión a Internet.
- **Consumir una API externa para obtener datos actualizados:** Integraré una API externa para obtener información actualizada sobre películas y series, garantizando que los datos estén siempre al día.
- **Utilizar un modelo de inteligencia artificial para generar preguntas de trivia:** Se utilizará un modelo de inteligencia artificial para generar preguntas sobre películas y series que se incluirán en el modo trivia, proporcionando contenido dinámico y desafiante.
- **Emplear un modelo de inteligencia artificial para moderación automática de comentarios:** Utilizando un modelo de inteligencia artificial, se moderarán los comentarios de los usuarios antes de su publicación, detectando lenguaje inapropiado y garantizando un entorno seguro y respetuoso para la comunidad.

2.3. Objetivos personales

Los objetivos personales se refieren a las metas y aspiraciones que tengo durante el proceso de creación de la aplicación. Estos objetivos no solo contemplan el fortalecimiento de mis habilidades técnicas, sino también el desarrollo de una mejor comprensión de las tecnologías utilizadas en el proyecto. Algunos de mis objetivos personales son:

- **Mejorar mis habilidades en programación *Android*:** A través de este proyecto, busco perfeccionar mis habilidades en el desarrollo de aplicaciones *Android* utilizando *Kotlin* y *Jetpack Compose*, dos tecnologías fundamentales en el desarrollo moderno de aplicaciones móviles.
- **Ampliar mis conocimientos en inteligencia artificial:** Este proyecto me permitirá aplicar modelos de inteligencia artificial en tareas como la generación de preguntas y la moderación de contenido, lo que contribuye a ampliar mis conocimientos en esta área.
- **Experimentar con tecnologías de API y su integración:** A través de este proyecto, busco mejorar mis habilidades en la integración de APIs en aplicaciones móviles, aprendiendo a consumir datos, manejar respuestas y adaptarlas a las necesidades de la aplicación.
- **Aplicar buenas prácticas en el desarrollo de aplicaciones:** Mi objetivo es seguir principios de desarrollo de software que mejoren la organización y calidad del código, como una estructura clara por capas, la realización de pruebas unitarias, el uso de herramientas de integración continua y la escritura de código limpio y mantenible.
- **Optimizar la experiencia del usuario en aplicaciones móviles:** Quiero crear una experiencia de usuario atractiva e intuitiva, poniendo el foco en mejorar la interfaz de usuario y la interacción de la aplicación para garantizar que los usuarios disfruten de una experiencia fluida y satisfactoria.
- **Continuar aprendiendo sobre la integración de servicios externos y funcionalidad *offline*:** A través de este proyecto, quiero seguir aprendiendo sobre cómo integrar servicios externos de manera eficiente y asegurar que la aplicación ofrezca funcionalidad *offline* robusta, lo que mejorará la experiencia del usuario en situaciones de baja conectividad.

- **Desarrollar la capacidad de gestionar grandes volúmenes de datos:** Este proyecto también me ayudará a mejorar mis habilidades en la gestión de grandes cantidades de datos, tanto en el lado del servidor como en la aplicación cliente, utilizando tecnologías como *Room* y una *base de datos* en la nube, lo que es esencial para manejar eficientemente el contenido de películas y series.

3. Conceptos teóricos

3.1. Arquitectura MVVM (*Model-View-ViewModel*)

El patrón arquitectónico **MVVM** proporciona una separación clara entre la lógica de la presentación, la gestión del estado y los datos. Esta división facilita el mantenimiento y escalabilidad de la aplicación, permitiendo que las vistas (UI) sean desacopladas de la lógica de negocio, y facilita la prueba de componentes como el *ViewModel* que maneja la lógica y el *Model* que gestiona los datos. En la sección 4.3 se ofrecen detalles sobre la aplicación de esta arquitectura al proyecto.

3.2. API REST

Una **API REST** (*Representational State Transfer*) [26] es una interfaz que facilita la comunicación entre sistemas a través de peticiones HTTP, donde los recursos se identifican mediante *URLs*. En esta aplicación, se ha integrado una API REST externa para acceder a datos actualizados sobre películas y series.

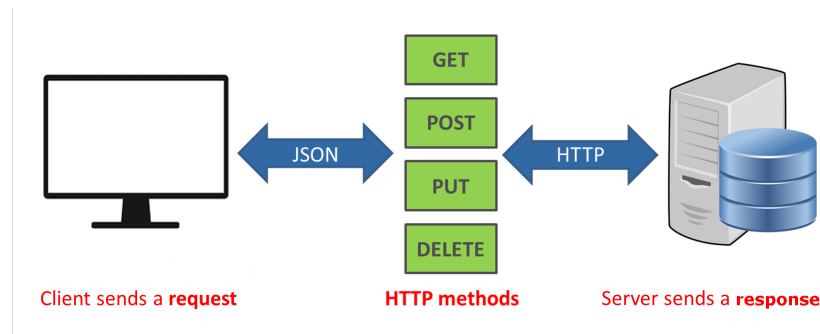


Figura 3.1: Esquema del funcionamiento de una API REST. Fuente: aprendiendoarduino.wordpress.com

El uso de APIs REST permite consultar grandes volúmenes de información en tiempo real, sin necesidad de almacenarla localmente. Las operaciones básicas que se pueden realizar son:

- **GET:** Recuperar recursos del servidor.
- **POST:** Enviar datos para crear nuevos recursos.
- **PUT/PATCH:** Modificar recursos existentes.
- **DELETE:** Eliminar recursos del servidor.

Los métodos *GET*, *POST*, *PUT*, y *PATCH* devuelven generalmente datos en formato *JSON*, lo que permite una fácil manipulación y presentación de la información en la aplicación. Este formato es ampliamente utilizado debido a su estructura sencilla y la compatibilidad con múltiples lenguajes de programación.

3.3. Corrutinas en *Kotlin*

Las **corrutinas** [19] son una característica de *Kotlin* que permite gestionar operaciones asíncronas y concurrentes de forma eficiente y con una sintaxis más sencilla que los enfoques tradicionales como *AsyncTask* o *Thread*. En el desarrollo de esta aplicación, las corrutinas se utilizaron ampliamente para acceder a la base de datos local con *Room*, manejar la comunicación con *Firestore* sin bloquear el hilo principal de la interfaz de usuario, y para manejar el cambio de conexión de manera fluida y reactiva.

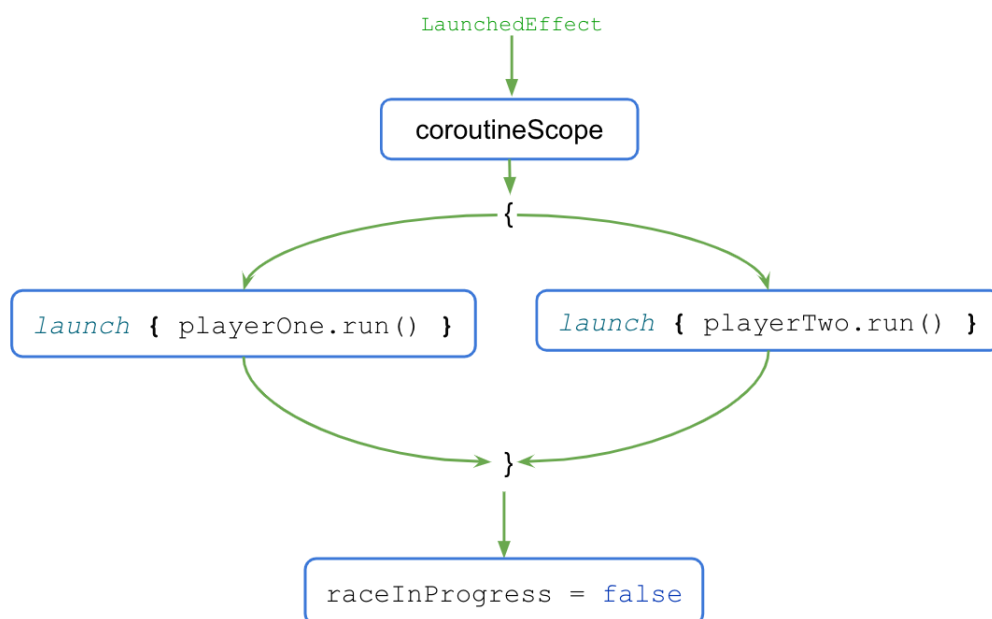


Figura 3.2: Esquema del funcionamiento de las corrutinas. Fuente: developer.android.com

El uso de corrutinas mejora la experiencia del usuario al mantener una interfaz fluida y reactiva, incluso mientras se realizan tareas intensivas en segundo plano. Gracias a las funciones `suspend`, es posible escribir código asíncronico de forma secuencial, facilitando su comprensión y mantenimiento.

Además, mediante el uso de *viewModelScope*, las corrutinas se ejecutan de forma segura dentro del ciclo de vida del *ViewModel*, evitando pérdidas de memoria o llamadas innecesarias cuando la pantalla ya no está activa. También se emplean estructuras como *Dispatchers.IO* para operaciones de entrada/salida y *withContext* para cambiar de contexto cuando es necesario.

Las corrutinas han sido fundamentales para gestionar correctamente la interacción entre el repositorio de datos, la base de datos local y la nube, logrando una arquitectura más reactiva, moderna y escalable.

3.4. Persistencia Local

La **persistencia local** es una característica clave en el desarrollo de aplicaciones móviles modernas, ya que permite almacenar y acceder a datos directamente en el dispositivo del usuario, incluso en ausencia de conexión a

internet. Esto garantiza una experiencia de usuario continua, permitiendo que la aplicación funcione correctamente sin depender exclusivamente de una conexión activa a un servidor remoto.

Existen varias formas de implementar la persistencia local en Android, entre las que destacan:

- ***SharedPreferences***: Permite almacenar pares clave-valor de forma sencilla y rápida, ideal para guardar configuraciones, estados simples o preferencias del usuario. No está diseñado para datos complejos o grandes volúmenes de información.
- **Archivos locales**: Se pueden usar archivos de texto, *JSON* o binarios para guardar información estructurada. Son útiles cuando se requiere controlar directamente el formato o acceder sin necesidad de una base de datos, aunque su uso puede volverse complejo al escalar.
- ***Room Database***: Es una herramienta moderna de Android Jetpack que hace más fácil trabajar con bases de datos *SQLite*. Con *Room*, puedes definir entidades, relaciones y consultas directamente en tu código *Kotlin* usando anotaciones, lo que simplifica mucho la interacción con bases de datos relacionales. Es ideal para almacenar grandes cantidades de datos organizados y realizar operaciones *CRUD* (Crear, Leer, Actualizar, Borrar) de forma eficiente y segura.

3.5. Base de Datos No Relacional

Las bases de datos no relacionales (*noSQL*) [33] son sistemas de gestión de bases de datos que no siguen el modelo tradicional de bases de datos relacionales. A diferencia de las bases de datos relacionales, que organizan los datos en tablas con filas y columnas, las bases de datos *NoSQL* permiten almacenar y gestionar datos de manera más flexible y escalable.

En lugar de seguir un esquema rígido con tablas fijas, las bases de datos *NoSQL* permiten almacenar datos de manera más flexible, ya sea no estructurados o semi-estructurados. Esto facilita su adaptación a diferentes tipos de aplicaciones y datos. Están diseñadas para ofrecer un alto rendimiento, ser escalables de forma horizontal y distribuir los datos entre varios servidores con facilidad.

Una de las principales ventajas de las bases de datos *NoSQL* es su capacidad para gestionar grandes volúmenes de datos, sobre todo cuando

3.6. GESTIÓN REACTIVA DEL ESTADO: *LIVEDATA* Y *STATEFLOW*

estos no tienen una estructura definida o pueden cambiar con el tiempo. Además, suelen ser ideales para aplicaciones que necesitan alta disponibilidad y un rendimiento rápido en operaciones de lectura y escritura.

3.6. Gestión Reactiva del Estado: *LiveData* y *StateFlow*

LiveData y ***StateFlow*** son herramientas fundamentales en el desarrollo de aplicaciones *Android* modernas para gestionar y monitorizar cambios en el estado de los datos de manera reactiva.

LiveData es una clase observable proporcionada por *Android Jetpack* que permite a la *UI* reaccionar automáticamente cuando cambian los datos. Se integra fácilmente con el ciclo de vida de los componentes de *Android* (como *Activity* o *Fragment*), evitando fugas de memoria y actualizaciones innecesarias. Su uso es común en combinación con los *ViewModel*, permitiendo que la interfaz de usuario observe los datos sin necesidad de lógica adicional.

En *Jetpack Compose*, para usar ***LiveData*** en composables, se emplea la extensión `observeAsState()`, que convierte un `LiveData<T>` en un `State<T>` observable. Esto permite que la *UI* se actualice automáticamente cuando el valor de ***LiveData*** cambia, integrándose de forma natural con el sistema reactivo de *Compose*.

StateFlow, por otro lado, forma parte de *Kotlin Flow* y es un tipo especial de *Flow* diseñado para mantener y emitir el estado actual de forma continua. Es más flexible y adecuado para arquitecturas modernas, especialmente cuando se utiliza junto con corrutinas y estructuras asíncronas. A diferencia de *LiveData*, *StateFlow* no está acoplado al ciclo de vida, por lo que su uso es más general y multiplataforma.

En *Jetpack Compose*, ***StateFlow*** se utiliza frecuentemente junto con la función `collectAsState()` para observar el estado dentro de composables y actualizar la *UI* automáticamente cuando el estado cambia, favoreciendo una interfaz reactiva y declarativa.

Ambos mecanismos permiten separar la lógica de presentación de los datos observados, fomentando una arquitectura más limpia y mantenible.

4. Técnicas y herramientas

4.1. Metodología de Desarrollo

Para la gestión del desarrollo se adoptó una metodología ágil basada en una adaptación de **Scrum** [28], que permitió planificar el trabajo de forma iterativa y mejorar progresivamente la aplicación según las necesidades detectadas y el feedback obtenido en cada *sprint*. Esta metodología es especialmente adecuada para proyectos con requisitos cambiantes, ya que facilita la rápida adaptación a nuevos requerimientos.

Para apoyar esta organización, se utilizó **GitHub** como plataforma central para el control de versiones y la gestión del proyecto. En particular, mediante **GitHub Projects** [6] se implementó la metodología *Scrum* dividiendo el trabajo en *sprints* y asignando tareas específicas. Esto permitió un seguimiento claro del progreso, manteniendo el enfoque en los objetivos de cada fase y consolidando tanto el código como la planificación en un único entorno, lo que favoreció un flujo de trabajo ordenado y eficiente.

4.2. Entorno de Desarrollo

En el desarrollo de la aplicación, se ha elegido **Android Studio** como el entorno de desarrollo integrado (*IDE*) principal. **Android Studio** es la herramienta oficial proporcionada por *Google* para la creación de aplicaciones *Android*, y es ampliamente utilizada debido a su robustez y conjunto de características diseñadas específicamente para el desarrollo móvil en la plataforma *Android* [8]. Esta elección se justifica por diversas razones clave:

- **Integración nativa con el ecosistema *Android*:** *Android Studio* está profundamente integrado con las bibliotecas y componentes oficiales de *Android*, como las bibliotecas de arquitectura (*Room*, *LiveData*, *ViewModel*, etc.), las herramientas de pruebas y depuración, y el emulador de *Android*. Esto permite una experiencia fluida y optimizada para los desarrolladores.
- **Soporte de *Kotlin* y *Java*:** El *IDE* ofrece un soporte excelente para ambos lenguajes de programación, aunque *Kotlin* es el principal lenguaje de desarrollo recomendado por *Google* para *Android*. La herramienta permite gestionar tanto el código en *Kotlin* como en *Java* sin complicaciones, lo que resulta en un entorno flexible y eficiente.
- **Emuladores y pruebas:** *Android Studio* cuenta con un emulador de *Android* avanzado que permite probar las aplicaciones en múltiples configuraciones de dispositivos y versiones de *Android*. Además, la integración de pruebas unitarias y pruebas de *UI* en *Android Studio* facilita un ciclo de desarrollo eficiente y un manejo adecuado de la calidad del código.
- **Integración con *Git* y herramientas colaborativas:** *Android Studio* optimiza el trabajo en equipo al ofrecer soporte nativo para sistemas de control de versiones como *Git*, facilitando la colaboración y la gestión eficiente del código fuente.

En cuanto al **lenguaje de programación**, se ha optado por ***Kotlin*** [18], un lenguaje moderno que es completamente compatible con el sistema *Android* y es el lenguaje oficial recomendado por *Google* para el desarrollo de aplicaciones en esta plataforma. Las razones para elegir *Kotlin* sobre *Java* son las siguientes:

- **Sintaxis más limpia y expresiva:** *Kotlin* permite escribir código más claro y conciso que *Java*. Gracias a sus características como las funciones de extensión, los tipos inmutables, las expresiones lambda y las funciones de orden superior, puedes escribir código de forma más simple sin perder legibilidad ni facilidad de mantenimiento.
- **Mayor seguridad contra errores comunes:** *Kotlin* está diseñado para evitar errores frecuentes en la programación, como los errores de punteros nulos (*NullPointerException*). Con su sistema de tipos nulos y no nulos, manejar los valores nulos es mucho más seguro, lo que reduce las posibilidades de fallos en tiempo de ejecución.

- **Total compatibilidad con Java:** *Kotlin* es completamente compatible con *Java*, lo que significa que puedes seguir utilizando las bibliotecas y *frameworks* escritos en *Java* sin ningún problema. Esto te permite aprovechar el gran ecosistema de *Android*, que históricamente ha sido desarrollado en *Java*, al mismo tiempo que disfrutas de las ventajas que *Kotlin* aporta.
- **Características avanzadas:** *Kotlin* ofrece funcionalidades modernas como la programación funcional, corrutinas para gestionar la concurrencia y la asincronía, y un manejo de datos simplificado. Estas herramientas son especialmente útiles para crear aplicaciones más eficientes, un aspecto clave en *Android*, donde la gestión de la concurrencia y la eficiencia es fundamental.
- **Mayor productividad y menos errores:** *Kotlin*, al ser un lenguaje más expresivo y contar con características como la inferencia de tipos, permite a los desarrolladores enfocarse más en la lógica de negocio y menos en los detalles técnicos como el manejo de tipos y excepciones. Esto no solo aumenta la productividad, sino que también reduce la posibilidad de cometer errores.
- **Ecosistema y soporte de Google:** Desde que *Google* adoptó *Kotlin* como el lenguaje preferido para el desarrollo en *Android*, ha crecido enormemente la comunidad de desarrolladores, ofreciendo más recursos, documentación y bibliotecas. *Kotlin* es también el lenguaje principal en herramientas como *Android Jetpack* y otras librerías modernas, lo que asegura un gran soporte a largo plazo.
- **Lenguaje de referencia oficial:** Desde mayo de 2019, *Kotlin* es el lenguaje de referencia de *Google* para el desarrollo de apps *Android*. Es más, *Google* y *JetBrains* crearon la *Kotlin Foundation* para desarrollar este lenguaje mano a mano.¹

4.3. Técnicas y Arquitectura

Una de las principales técnicas utilizadas en el desarrollo de la aplicación es el uso de la arquitectura MVVM (*Model-View-ViewModel*), que se emplea para separar de manera clara y eficiente la lógica de negocio de la interfaz de usuario. El patrón MVVM facilita la creación de aplicaciones escalables

¹<https://hipertextual.com/2019/06/kotlin-lenguaje-android-java>

y mantenibles al permitir que la lógica de la interfaz de usuario se gestione de manera reactiva y autónoma mediante el uso de **ViewModels** [10].

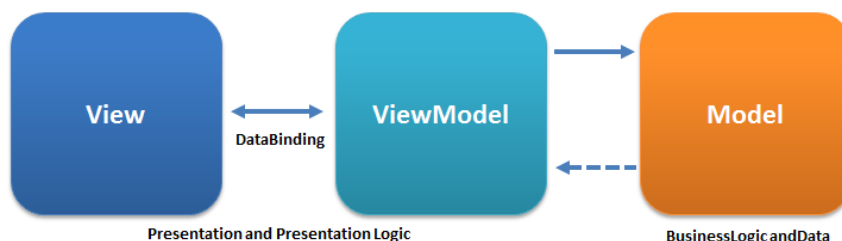


Figura 4.1: Esquema de la estructura MVVM

- **Model:** Representa los datos y la lógica de negocio de la aplicación. En el caso de este proyecto, los datos se gestionan mediante **Room** para la persistencia local y **Firestore** para la sincronización remota de los datos del usuario.
- **View:** Se encarga de mostrar la interfaz de usuario y de interactuar con el usuario. En este proyecto, se utiliza **Jetpack Compose** para construir interfaces dinámicas y reactivas, que se actualizan automáticamente en respuesta a cambios en los datos.
- **ViewModel:** Actúa como intermediario entre el *Model* y la *View*. Se encarga de gestionar el estado de la *UI* y de exponer los datos que deben mostrarse en la interfaz de usuario. Los **ViewModels** permiten que la lógica de negocio se mantenga separada de la interfaz de usuario, mejorando la modularidad y la prueba unitaria de la aplicación.

El uso de MVVM en combinación con herramientas como **LiveData** y **StateFlow** permite una gestión eficiente del estado y de la lógica de la interfaz. Gracias a la arquitectura MVVM, la *UI* se actualiza automáticamente cuando los datos cambian, mejorando la experiencia de usuario y reduciendo la necesidad de manejar manualmente los estados de la aplicación.

4.4. Diseño de Interfaz de Usuario

Una de las decisiones más importantes que se tomaron en el desarrollo de la interfaz de usuario fue la elección de **Jetpack Compose**, el nuevo *framework* declarativo de *Android* para diseñar interfaces [11]. Opté por

esta tecnología en lugar del enfoque tradicional basado en *XML* por varias razones clave:

- **Mayor flexibilidad y simplicidad:** *Jetpack Compose* ofrece un enfoque más flexible para construir interfaces. Mientras que *XML* requiere de múltiples archivos y jerarquías complejas para definir los elementos de la *UI*, *Jetpack Compose* permite hacerlo de manera más directa y concisa en el propio código *Kotlin*, lo que resulta más fácil de gestionar y mantener.
- **Integración natural con *Kotlin*:** *Jetpack Compose* está diseñado para integrarse de manera fluida con *Kotlin*, lo que facilita mucho la escritura de código y mejora la productividad. Dado que *Kotlin* es el lenguaje principal del proyecto, esta integración natural permite una experiencia de desarrollo más coherente y menos propensa a errores.
- **Manejo eficiente del estado:** Una de las características que más me convencieron de *Jetpack Compose* es su capacidad para gestionar el estado de la *UI* de manera más sencilla y eficiente. Esto me ha permitido construir pantallas dinámicas que se adaptan de manera reactiva a los cambios, sin necesidad de manejar complejos mecanismos de actualización de la *UI* como en *XML*.
- **Desarrollo más ágil y rápido:** Con *Jetpack Compose*, se pudo desarrollar interfaces de usuario de manera más rápida, sin tener que lidiar con la redundancia de código que a veces ocurre con *XML*. Además, la estructura declarativa facilita ver de manera más clara cómo se van a comportar los elementos de la *UI*, lo que me permitió iterar y modificar diseños de forma más ágil.
- **Mejor mantenimiento y escalabilidad:** Gracias a su integración con herramientas como *ViewModel* y otras bibliotecas de *Jetpack*, *Jetpack Compose* contribuye a una arquitectura más limpia y escalable. Esto es algo que considero más complicado de lograr en *XML* debido a la complejidad de gestionar el ciclo de vida de la *UI* de manera eficiente.

4.5. Obtención de Datos

Durante el desarrollo del proyecto, se realizó un análisis exhaustivo de las distintas fuentes de datos disponibles para obtener información actualizada

y fiable sobre películas y series. El objetivo principal era integrar una fuente que ofreciera un catálogo amplio, actualizado con frecuencia, y que fuera accesible mediante una API moderna y bien documentada.

Finalmente, se seleccionó la API de *TMDB* (The Movie Database) [30], una plataforma reconocida por su extensa base de datos de contenidos audiovisuales, su comunidad activa de colaboradores y su documentación clara. *TMDB* proporciona una interfaz *RESTful* que permite realizar peticiones *HTTP* para consultar detalles sobre películas, series, actores, géneros, valoraciones, imágenes promocionales, *trailers*, fechas de estreno, entre otros datos relevantes.

Se valoraron otras alternativas como la API de *IMDb* [16], ampliamente conocida en el sector audiovisual. Sin embargo, se detectaron varias limitaciones importantes en esta opción: el acceso es más restringido, la versión gratuita es menos completa, y su integración requiere más configuraciones, además de políticas más estrictas respecto al uso comercial o público.

En cambio, *TMDB* ofrece un modelo de acceso abierto bajo registro, facilita el uso educativo y de desarrollo personal, y cuenta con bibliotecas auxiliares en distintos lenguajes de programación, lo cual facilitó su incorporación en la aplicación tanto desde el lado de la app *Android* como desde el *backend* implementado en *Python* [5]. En este proyecto, *Python* es el encargado de gestionar la actualización de los datos en la base de datos remota de *Firestore*, extrayendo la información desde *TMDB* mediante peticiones y organizándola según las necesidades de la aplicación.

Gracias a esta API, los usuarios pueden explorar contenido actualizado, descubrir las últimas tendencias, filtrar por sus géneros favoritos y acceder a información detallada que mejora su experiencia dentro de la aplicación.

4.6. Almacenamiento de Datos

Para el almacenamiento de datos del usuario, se ha utilizado *Firebase Firestore*, una base de datos *NoSQL* no relacional en tiempo real [9]. *Firestore* permite una sincronización inmediata de los datos con la aplicación, lo que mejora la experiencia del usuario en cuanto a rapidez y actualización de la información. Además, *Firestore* organiza los datos en **colecciones** y **documentos**, lo que proporciona una estructura flexible y escalable para el almacenamiento de los datos.

Para guardar los datos localmente en el dispositivo, se usa *Room* [13], que es una librería para manejar bases de datos en *Android*. *Room* hace que

trabajar con *SQLite* sea mucho más fácil, porque actúa como una capa que simplifica todo el proceso. Con solo usar algunas anotaciones en el código *Kotlin*, se puede crear y gestionar bases de datos, entidades y sus relaciones de forma más rápida y sin complicaciones.

En la aplicación, *Room* se utiliza para almacenar localmente información sobre películas y series, permitiendo un acceso rápido y eficiente a los datos incluso sin conexión a *Internet*. Además, se decidió almacenar solo un subconjunto limitado de resultados, guardando las **primeras 10 películas o series** de cada lista visible en pantalla, optimizando así el uso de recursos.

Asimismo, para el almacenamiento de imágenes de perfil en un servidor externo, se optó por una solución basada en *Imgur* [17], descartando el uso de *Firebase Storage* debido a que requería un plan de pago para permitir la carga y gestión de imágenes de forma adecuada. Las imágenes se suben directamente a *Imgur*, y la URL pública obtenida se guarda en *Firestore*.

Firestore: Firestore y autenticación

Se utilizó *Firestore* de *Firebase* como base de datos en la nube para almacenar los datos de los usuarios, sus preferencias y la información relacionada con las películas y series recomendadas.

Firestore facilitó la sincronización de datos entre el *backend* y el dispositivo, así como la gestión de usuarios mediante autenticación por correo electrónico y contraseña.

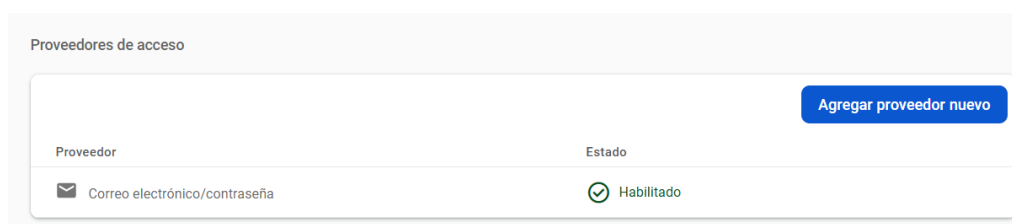


Figura 4.2: Método de acceso para los usuarios

Además, una de las características más destacadas de *Firestore* es su capacidad para realizar consultas en tiempo real. Esto permitió que las recomendaciones y las actualizaciones de los comentarios se reflejaran instantáneamente en la aplicación.

4.7. Inteligencia Artificial

Una de las decisiones más importantes que se tomaron en el proyecto fue integrar inteligencia artificial mediante ***Gemini*** para analizar y moderar los comentarios de los usuarios antes de que se publiquen, así como para generar las preguntas y la retroalimentación utilizadas en la sección de Trivia.

Desde el comienzo del proyecto se estudiaron diferentes alternativas de herramientas de procesamiento de lenguaje natural. Se analizaron opciones como *Perspective API* [20], *ChatGPT* [23] y *Gemini* [12], comparando su precisión, rapidez y facilidad de integración. Aunque *ChatGPT* ofrecía una gran capacidad de entendimiento y resultados aceptables en la detección de lenguaje inapropiado, finalmente me decanté por ***Gemini*** por varias razones clave:

- **Mayor precisión y comprensión del contexto:** *Gemini* demostró una capacidad superior para interpretar los comentarios dentro de su contexto, lo cual fue esencial para evitar moderaciones injustas o innecesarias.
- **Mejor integración con el proyecto:** Su compatibilidad con *Kotlin* y el entorno de desarrollo *Android Studio* me permitió incorporarlo fácilmente en la app, mejorando el flujo de trabajo general.
- **Rendimiento rápido y eficiente:** Como necesitaba respuestas casi instantáneas para ofrecer una buena experiencia al usuario, la rapidez de *Gemini* fue determinante.
- **Limitaciones de *ChatGPT*:** Aunque valoré mucho su capacidad, el límite de peticiones en la API de *ChatGPT* dificultaba su uso continuado en una app con múltiples usuarios. *Gemini* ofreció una solución más estable y escalable para este tipo de uso intensivo.

4.8. Desarrollo de la Memoria y Anexos

Para redactar y estructurar la memoria y los anexos del proyecto, se utilizó la plataforma en línea ***Overleaf*** [24], basada en $\text{\LaTeX}$. Se eligió esta herramienta por varias razones clave: su facilidad de uso, su potencia para el manejo de documentos técnicos y su capacidad de mantener un formato limpio y profesional en todo momento.

Desde el inicio, ***Overleaf*** ofreció un entorno de trabajo claro y ordenado. Gracias a su compatibilidad con $\text{\LaTeX}$, se pudo gestionar fácilmente

elementos como las citas y la bibliografía, generar índices, e insertar figuras, tablas, anexos y referencias cruzadas, todo con un formato uniforme y sin tener que preocuparme por el diseño.

A pesar de que el desarrollo del proyecto y la redacción del documento se realizaron de forma individual, **Overleaf** fue especialmente útil para facilitar la revisión del contenido por parte de mi tutora. Al tratarse de una herramienta colaborativa, fue posible compartir el enlace del documento, permitiendo el acceso en cualquier momento para realizar comentarios o sugerencias directamente sobre el texto, lo que agilizó notablemente el proceso de retroalimentación.

Además, la posibilidad de compilar el documento directamente desde el navegador y exportarlo en formato *PDF* ayudó a visualizar el resultado final en todo momento, permitiendo detectar rápidamente errores de formato o contenido. Esta flexibilidad y control sobre el documento final resultó fundamental para cumplir con los requisitos de presentación de la memoria y de los anexos.

4.9. Prototipado del Esquema de Navegación

Antes de comenzar con la programación de la aplicación, se definió cuidadosamente su diseño y estructura. Para ello, se utilizó la plataforma de diseño colaborativo en línea **Figma** [3], junto con **FigJam**, una herramienta complementaria destinada a crear diagramas, esquemas e ideas de forma visual. Ambas resultaron muy útiles para dar forma a la idea inicial del proyecto.

Figma es ampliamente utilizada en el mundo del diseño de interfaces debido a que permite crear prototipos interactivos y trabajar en equipo desde cualquier lugar gracias a su funcionamiento en la nube. *FigJam*, por su parte, es ideal para organizar ideas de forma rápida y visual, similar a una pizarra digital.

Con estas herramientas, se elaboró un *prototipo visual* de la aplicación que facilitó la visualización de cómo debía verse y funcionar. En *FigJam* se esquematizó la estructura de las pantallas, se definió la jerarquía de los elementos y se construyó el primer prototipo. Aunque durante el desarrollo se realizaron varios cambios, este diseño inicial fue fundamental para contar con una guía clara desde el principio y alinear las ideas principales del proyecto (véase la Figura 4.3).

También con *FigJam* se creó un *esquema de navegación* que representa todas las pantallas de la aplicación y cómo se conectan entre sí. Esta vista general permitió organizar la navegación de forma lógica y coherente, asegurando una experiencia fluida para el usuario. Contar con este esquema desde el inicio facilitó la detección temprana de posibles fallos de usabilidad antes de comenzar la programación, lo que ayudó a ahorrar tiempo y esfuerzo durante las etapas posteriores del desarrollo (véase la Figura 4.4).

TODAS LAS PANTALLAS SON SCROLLABLES
(VERTICAL, HORIZONTAL O AMBAS)

TODAS LAS PELÍCULAS/SERIES SON CLICKABLES

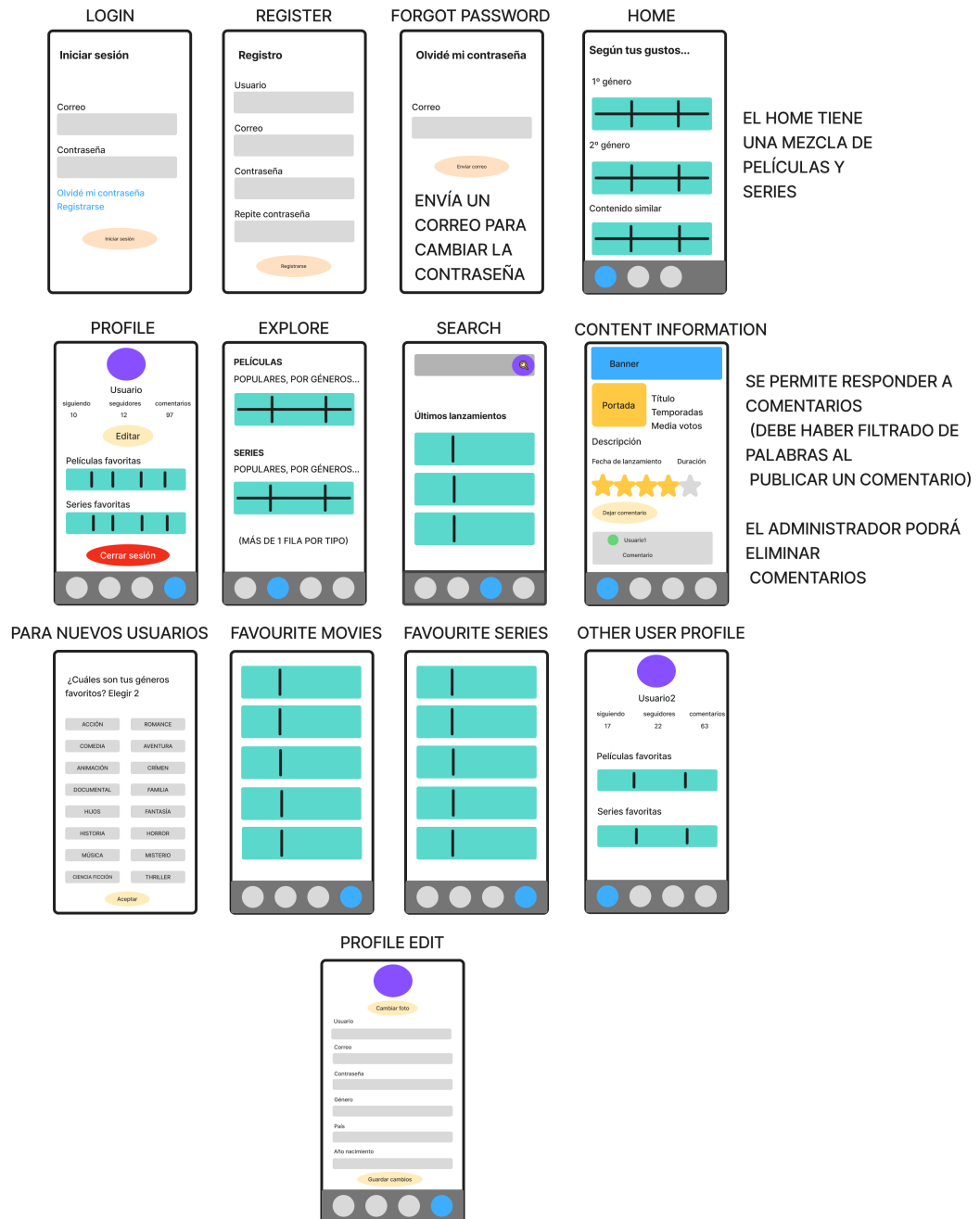


Figura 4.3: Prototipo inicial de la aplicación

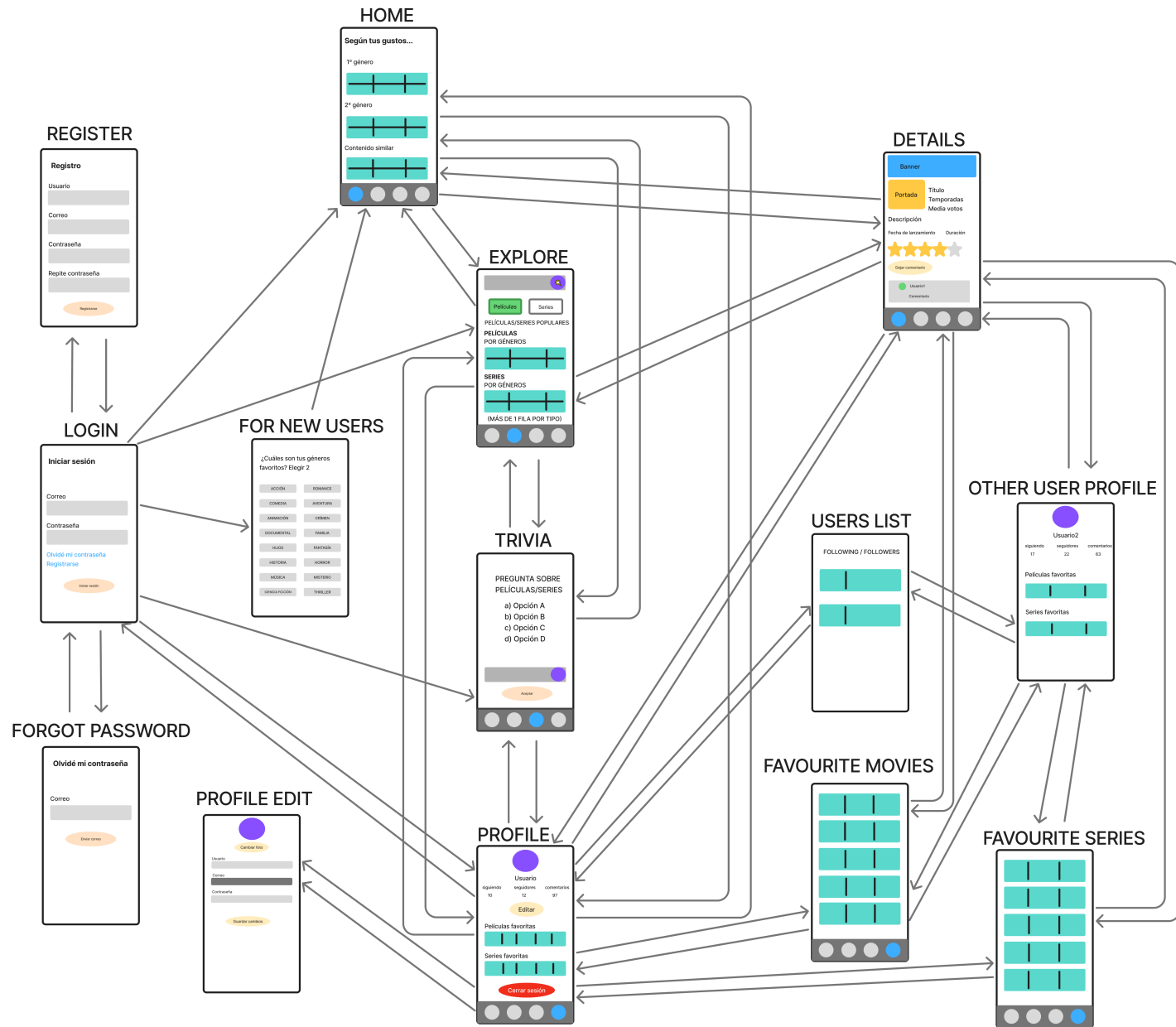


Figura 4.4: Esquema de navegación de la aplicación

Durante el desarrollo, uno de los objetivos principales fue mantener una calidad de código alta y consistente. Para medirla, se integró **SonarCloud** [29] en el flujo de trabajo, una herramienta muy útil para detectar automáticamente errores, posibles vulnerabilidades y problemas relacionados con la mantenibilidad del proyecto. Gracias a sus análisis estáticos, el código pudo ser revisado de forma objetiva para tomar decisiones fundamentadas sobre cómo mejorarlo.

La integración de **SonarCloud** se realizó utilizando **Gradle** con **Kotlin DSL (KTS)** [14], dado que la mayor parte del proyecto está desarrollado en *Kotlin*. Esta configuración permitió mantener el archivo de construcción limpio, ordenado y fácil de mantener. Fue un proceso bastante directo, y el uso de *Kotlin DSL* facilitó mucho la comprensión y personalización de la configuración necesaria para vincular correctamente el análisis con el repositorio.

Además, para evitar ejecutar manualmente los análisis tras cada cambio, se incorporó **GitHub Actions** [7]. Con esta automatización, cada vez que se sube código al repositorio, se lanza automáticamente un análisis de **SonarCloud**. Esto permite detectar problemas en una etapa temprana del desarrollo y asegurar que la calidad del código se mantiene de forma constante a lo largo del tiempo, sin dejar lugar a descuidos.

El panel y resultados de **SonarCloud** pueden consultarse públicamente en el siguiente enlace: https://sonarcloud.io/summary/overall?id=ivaanestepar_FlixFinderTV&branch=master.

Gracias a este enfoque, se ha logrado mantener un código más limpio, fiable y fácil de escalar, lo cual ha sido clave para la estabilidad y evolución del proyecto. El *dashboard* del proyecto se muestra en la figura 4.5.

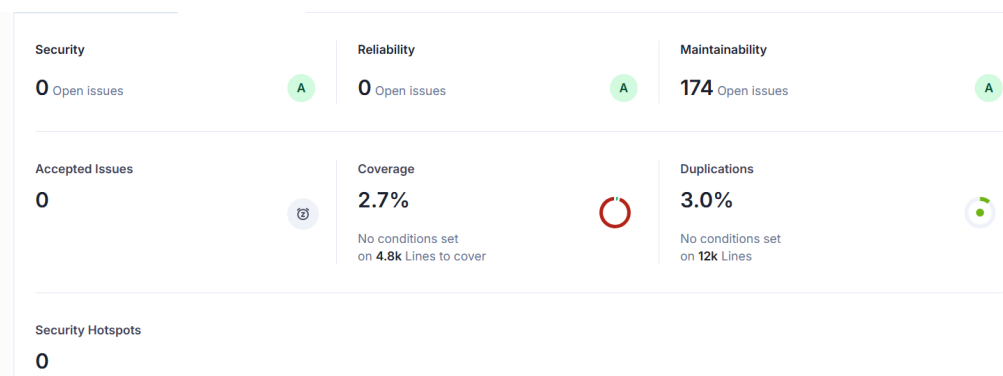


Figura 4.5: Análisis de calidad del código en *SonarCloud*

En la tabla 4.1 se ofrece un resumen de las herramientas y tecnologías empleadas en cada parte principal del proyecto: el propio desarrollo de la *app*, el API de comunicación con el *backend*, la persistencia de datos y la elaboración de la documentación.

Tabla 4.1: Herramientas y tecnologías utilizadas en cada parte del proyecto

Herramientas	<i>Android</i>	APIs	Persistencia	Documentación
Android Studio	X			
Room	X		X	
Jetpack Compose	X			
TMDB API		X		
Gemini API		X		
Firebase			X	
Kotlin	X			
Python	X		X	
Gradle	X			
SonarCloud	X			
JUnit	X			
MockK	X			
Corrutinas	X			
LiveData	X			
ViewModel	X			
Navigation Component	X			
Room DAO	X		X	
Firebase Firestore			X	
Firebase Authentication			X	
GitHub	X		X	X
Overleaf				X
Figma				X
Appetize.io	X			
Android YouTube Player	X			

5. Aspectos relevantes del desarrollo del proyecto

5.1. Metodologías Aplicadas

Dado que el desarrollo de la aplicación fue realizado de forma individual, se optó por una metodología ágil adaptada a un solo desarrollador, basada en una variante personalizada de **Scrum**. Aunque trabajé de manera independiente, el uso de **Scrum** me permitió estructurar el trabajo en ciclos cortos y entregas incrementales, lo que facilitó la planificación y organización de las tareas.

Dividí las tareas en *sprints* de dos semanas, con objetivos claramente definidos al inicio de cada ciclo. Al finalizar cada *sprint*, revisaba el avance y ajustaba las funcionalidades o el enfoque según las necesidades del proyecto. A lo largo del desarrollo, las tutorías con mi tutora se realizaban cada dos semanas, lo que me ofrecía la oportunidad de recibir retroalimentación directa sobre el progreso del proyecto y las decisiones tomadas. Estas tutorías fueron clave para realizar ajustes rápidos y asegurarme de que el proyecto estuviera alineado con los objetivos y las mejores prácticas.

El control de versiones y la gestión de tareas se hicieron utilizando **GitHub**, lo que facilitó la organización del código y el seguimiento del progreso. Además, **GitHub** permitió integrar herramientas de análisis de calidad como **SonarCloud**, lo que ayudó a garantizar que el código cumpliera con los estándares de calidad establecidos.

De este modo, la metodología ágil, combinada con las tutorías periódicas, me permitió gestionar el desarrollo de manera eficiente, asegurando la

entrega continua de mejoras y adaptaciones de acuerdo con las necesidades del proyecto y el *feedback* recibido.

5.2. Inicio del Proyecto

El proyecto actual surgió a partir de una aplicación *Android* que desarrollé durante mi experiencia Erasmus, la cual despertó mi interés por el desarrollo de aplicaciones móviles. Motivado por esa experiencia, decidí emprender el desarrollo de una nueva aplicación centrada en el descubrimiento y la recomendación de películas y series personalizadas según los gustos del usuario.

Esta aplicación recibió el nombre de **FlixFinderTV**.



Figura 5.1: Logo de la aplicación

La fase inicial del proyecto consistió en un análisis detallado de los requisitos y la definición de los objetivos principales. Durante esta fase, se establecieron funcionalidades clave como la recomendación de contenido personalizado, la búsqueda y filtrado por géneros, la interacción a través de comentarios moderados, y el acceso *offline*, lo que garantizaría una experiencia completa para el usuario.

Estas decisiones iniciales fueron clave para crear una base sólida, que haga posible que la aplicación crezca y se siga optimizando conforme se agreguen nuevas funcionalidades.

5.3. Frontend

El desarrollo del *frontend* de la aplicación se llevó a cabo utilizando **Jetpack Compose**, la moderna herramienta de UI para aplicaciones Android. La principal ventaja de **Compose** es su capacidad para crear interfaces de usuario de manera declarativa y con un control total sobre el estado de la *UI*, lo que facilita el desarrollo de aplicaciones reactivas y dinámicas.

Para la estructura del frontend, se adoptó la arquitectura **MVVM** (*Model-View-ViewModel*), que permite una separación clara entre la lógica de negocio y la interfaz de usuario. Los datos de las películas y series se manejan en **ViewModels**, los cuales obtienen los datos directamente de la base de datos local o externa, y luego pasan esos datos a las vistas para ser mostrados al usuario.

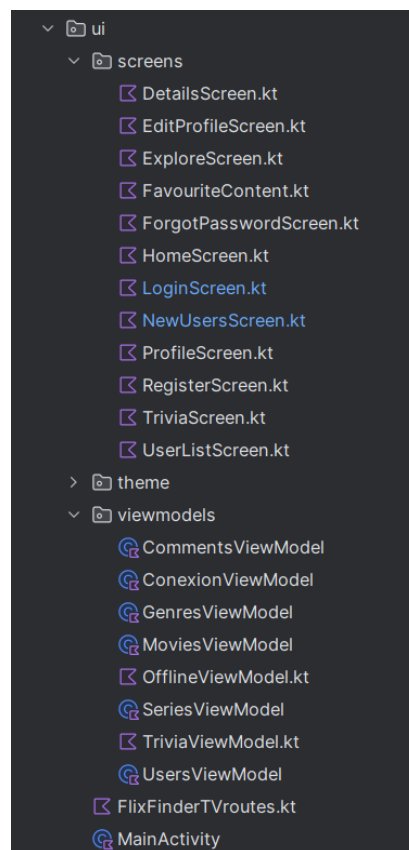


Figura 5.2: Organización de carpetas en el frontend

Se diseñaron pantallas clave como la pantalla de **Home**, que muestra los próximos lanzamientos y las películas o series recomendadas al usuario

según sus gustos, utilizando la información de sus géneros favoritos; la pantalla de **Details**, donde se presentan más datos sobre una película o serie seleccionada, incluyendo su sinopsis, valoración, tráiler y los comentarios realizados por otros usuarios; la pantalla de **Explore**, que permite a los usuarios descubrir nuevo contenido mediante la exploración por géneros e intereses, así como buscar por título o filtrar por género y tipo; y la pantalla de **Profile**, donde los usuarios pueden visualizar su información personal y estadísticas básicas.

Asimismo, se desarrollaron pantallas adicionales con funcionalidades específicas: la pantalla de **Edit Profile** permite modificar el nombre de usuario y la foto de perfil, actualizando los datos almacenados en Firestore; la pantalla de **Favourite Content** muestra dos listas personalizadas con el contenido marcado como favorito por el usuario, para un acceso más rápido. En esta sección se pueden añadir como máximo 20 películas y 20 series, lo que permite mantener una experiencia ágil y centrada en los títulos más relevantes para el usuario; la pantalla de **Forgot Password** permite al usuario restablecer su contraseña a través de su correo electrónico registrado; y la pantalla de **Login** permite gestionar el acceso utilizando **Firestore Authentication**, permitiendo entrar a las funcionalidades personalizadas de la aplicación.

Además, se incluye la pantalla de **New Users**, mostrada únicamente al iniciar sesión por primera vez, donde el usuario elige sus dos géneros preferidos para personalizar las recomendaciones desde el inicio; la pantalla de **Register** facilita la creación de una nueva cuenta con correo electrónico y contraseña; la pantalla de **Trivia** ofrece un juego interactivo de preguntas y respuestas sobre cine y series, con retroalimentación; y finalmente, la pantalla de **User List**, que permite al usuario ver y gestionar a las personas que sigue o que lo siguen, favoreciendo así la interacción social dentro de la app.

Cabe destacar que el **Trivia** está traducido según el idioma del dispositivo: si el dispositivo está configurado en español, se mostrará en español; si está en otro idioma, se mostrará en inglés.

Todo esto se manejó mediante **Jetpack Navigation** para gestionar de forma eficiente y estructurada la navegación entre pantallas dentro de la aplicación.

En la Figura 5.2 también se observan los distintos *ViewModels* utilizados en la aplicación. Como mencioné anteriormente, se optó por el uso de *ViewModels* como parte del patrón de arquitectura **MVVM** para separar claramente la lógica de negocio y de datos de la interfaz de usuario. Gracias

a los *ViewModels*, cada pantalla puede acceder a los datos necesarios de forma reactiva, observando cambios a través de **LiveData** o **StateFlow**, sin depender directamente de la capa de almacenamiento o red.

Cada *ViewModel* tiene un propósito específico, lo que permite una gestión más clara y eficiente del código, facilitando su mantenimiento y evolución a lo largo del tiempo.

5.4. *Backend*

El backend del proyecto se centra principalmente en la integración de **Firestore** y **Room**.

Estructura de la base de datos en *Firestore*

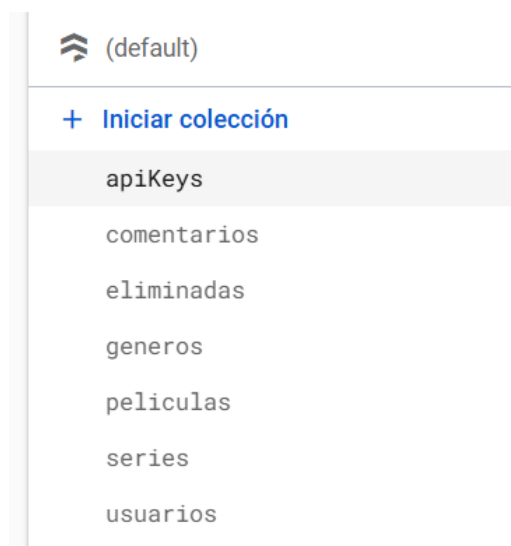


Figura 5.3: Organización de carpetas en *Firestore*

En la Figura 5.3 se muestran las colecciones del proyecto en *Firestore*, donde:

- **apiKeys**: almacena la *apiKey* de TMDB.
- **comentarios**: almacena los comentarios y respuestas de las películas/series.

- **eliminadas**: muestra las películas y series que se han eliminado en cada actualización de la base de datos.
- **generos**: almacena los géneros junto con su *id*.
- **peliculas**: almacena las películas.
- **series**: almacena las series.
- **usuarios**: almacena los datos de los usuarios.

Seguridad en el almacenamiento de claves API

En la colección `apiKeys` se almacena la `apiKey` de TMDb de forma segura. Para ello, primero se cifra utilizando el algoritmo **Fernet** [2], que forma parte del módulo `cryptography` de *Python*.

Fernet proporciona cifrado simétrico autenticado basado en *AES* en modo *CBC* con una clave de 128 bits, junto con una firma *HMAC* para garantizar la integridad del mensaje. La clave cifrada se codifica en `base64` para su almacenamiento como cadena de texto en *Firestore*.

Por otro lado, las claves API utilizadas exclusivamente por la aplicación móvil se guardan de forma segura en el archivo `local.properties`, lo cual evita exponerlas directamente en el código fuente.

Servicio de actualización de *Firestore*

Para mantener actualizada la base de datos de *Firestore* con la información más reciente de películas y series, se ha desarrollado un servicio en *Python* encargado de realizar dichas actualizaciones.

Este servicio automatiza la integración con la API de *The Movie Database* para obtener datos recientes y gestionar los cambios en *Firestore*.

El proceso principal del servicio consiste en eliminar las **cinco películas y series más antiguas** almacenadas y reemplazarlas por las **cinco películas y series más nuevas** obtenidas de las **primeras páginas** de resultados de la API, ya que son las que se actualizan con mayor frecuencia.

Para ello, utiliza la librería oficial de *Firebase* para *Python*, `firebase-admin`, que permite autenticar y realizar operaciones seguras sobre la base de datos en la nube.

Esta estrategia permite mantener una colección actualizada y limitada en tamaño, evitando la acumulación de datos obsoletos y garantizando la **coherencia** y **fiabilidad** de la información almacenada.

Además, el servicio está completamente automatizado mediante el uso de **GitHub Actions**, programado para ejecutarse automáticamente cada **lunes a las 09:00 UTC**. Esto garantiza una actualización periódica sin necesidad de intervención manual, asegurando que la aplicación disponga siempre de datos frescos y actualizados, mejorando así el rendimiento y la calidad de las recomendaciones y el descubrimiento de contenido audiovisual.

Room

Para la gestión de la persistencia local de datos en la aplicación, se utilizó *Room*, una biblioteca de persistencia de *Android* que actúa como una capa de abstracción sobre *SQLite*. Esta librería proporciona una API más intuitiva y segura, permitiendo definir entidades, relaciones y consultas *SQL* a través de anotaciones en código *Kotlin*, lo cual facilita el mantenimiento y escalabilidad del proyecto.

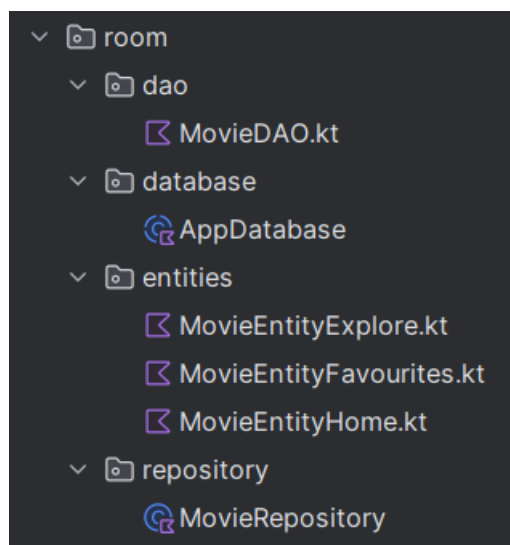


Figura 5.4: Organización de carpetas en *Room*

Como se observa en la Figura 5.4, la imagen muestra la organización de los archivos relacionados con *Room* dentro del proyecto. Se puede observar la presencia de las siguientes clases clave:

- **MovieDAO**: contiene las funciones de acceso a datos (consultas, inserciones y eliminaciones) para cada una de las entidades gestionadas en la base de datos.

- ***AppDatabase***: representa la base de datos principal del proyecto y está anotada con **@Database**. Integra todas las entidades y proporciona el punto de acceso para obtener una instancia del *DAO*.
- ***MovieEntityExplore***, ***MovieEntityFavourites*** y ***MovieEntityHome***: cada una de estas clases representa una entidad diferente dentro de *Room*, lo que permite gestionar de forma independiente las listas de películas y series relacionadas con la sección de exploración, favoritos y pantalla principal.
- ***MovieRepository***: actúa como intermediario entre la base de datos y las capas superiores de la arquitectura (como los *ViewModels*). Centraliza las operaciones sobre todas las entidades, asegurando una gestión coherente de los datos locales.

En el contexto de esta aplicación, *Room* fue implementado para almacenar localmente información relacionada con películas y series clasificadas por distintos géneros. Se definieron múltiples entidades, cada una correspondiente a una categoría específica, y un único repositorio centralizado (*MovieRepository*) se encargó de coordinar las operaciones de inserción, consulta y eliminación sobre la base de datos local.

Con el objetivo de mejorar el rendimiento de la aplicación, reducir el consumo de recursos y evitar la sobrecarga de datos, se decidió almacenar únicamente un subconjunto limitado de resultados. Específicamente, solo se guardaron las primeras **10 películas o series** de cada lista visible en cada pantalla, excepto en la sección de favoritos, donde se permite almacenar un máximo de **20 elementos**. Esta decisión fue tomada considerando que la mayoría de los usuarios interactúan principalmente con los primeros elementos presentados, y que una sincronización completa de todos los datos podría ser innecesaria y costosa en términos de almacenamiento.

5.5. Modo *Offline*

Una de las características clave de la aplicación es el soporte para el uso *offline*. La aplicación permite a los usuarios acceder a las recomendaciones de películas y series, así como a su historial y preferencias, incluso sin conexión a internet. Para esto, se implementó el almacenamiento local utilizando **Room**, una base de datos que almacena los datos esenciales en el dispositivo del usuario.

Para comprobar si el dispositivo tiene conexión a internet, se realiza una solicitud HTTP a `google.com`. Si la conexión se realiza con éxito y se recibe una respuesta HTTP válida, la función devuelve `true`. En caso contrario, se establece en `false`, lo que permite que la aplicación gestione la operación *offline*.

```
// Función para comprobar si hay acceso a Internet
suspend fun isInternetAvailable(): Boolean {
    return try {
        withContext(Dispatchers.IO) {
            val url = URL(spec: "https://www.google.com")
            val connection = url.openConnection() as HttpURLConnection
            connection.connectTimeout = 5000
            connection.readTimeout = 5000

            try {
                connection.connect() // Intentar la conexión
                connection.responseCode == HttpURLConnection.HTTP_OK
            } catch (e: Exception) {
                false // Si hay un error de conexión, devolver false
            }
        }
    } catch (e: Exception) {
        false // Si no se puede realizar la prueba, devolver false
    }
}
```

Figura 5.5: Función que comprueba la conexión

Cuando el usuario tiene conexión *Internet*, la aplicación sincroniza los datos con la base de datos en la nube, asegurando que la información esté siempre actualizada. Sin embargo, cuando la conexión no está disponible, el usuario puede seguir interactuando con la aplicación sin interrupciones, gracias a los datos almacenados localmente. Esta funcionalidad mejora la

experiencia del usuario, ya que no depende de una conexión constante para acceder al contenido.

5.6. Despliegue

Una vez finalizado el desarrollo y superadas las fases de prueba, se procedió al despliegue de la aplicación con el objetivo de permitir su visualización y evaluación de forma sencilla, sin necesidad de instalación directa en dispositivos físicos.

Para ello, se utilizó la plataforma **Appetize.io** [1], un servicio *online* que permite emular dispositivos *Android* e *iOS* directamente desde el navegador. Esta herramienta resultó útil para compartir la aplicación una vez finalizada, ya que ofrecía una experiencia interactiva similar a la de un dispositivo real, sin requerir configuraciones complicadas ni instalaciones adicionales.

El proceso de despliegue consistió en la generación del archivo *APK* de la aplicación desde *Android Studio*. Una vez generado, este archivo se subió a *Appetize*, donde se configuró una instancia de emulación para dispositivos *Android*. *Appetize* proporcionó un enlace público que permitió acceder a la aplicación de forma inmediata. Esta opción fue especialmente conveniente para la entrega y demostración del proyecto, garantizando accesibilidad multiplataforma y una experiencia fluida.

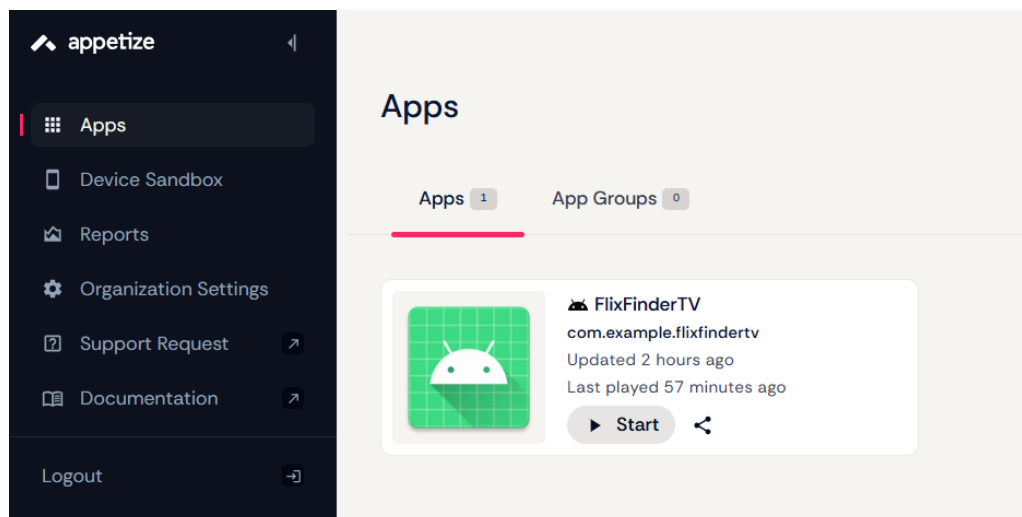


Figura 5.6: Aplicaciones guardadas en *Appetize*

Además, *Appetize* permitió establecer diferentes configuraciones para el entorno de ejecución, como la versión del sistema operativo, el tamaño de pantalla o el tipo de dispositivo emulado, lo que facilitó pruebas en condiciones similares a las de un entorno real.

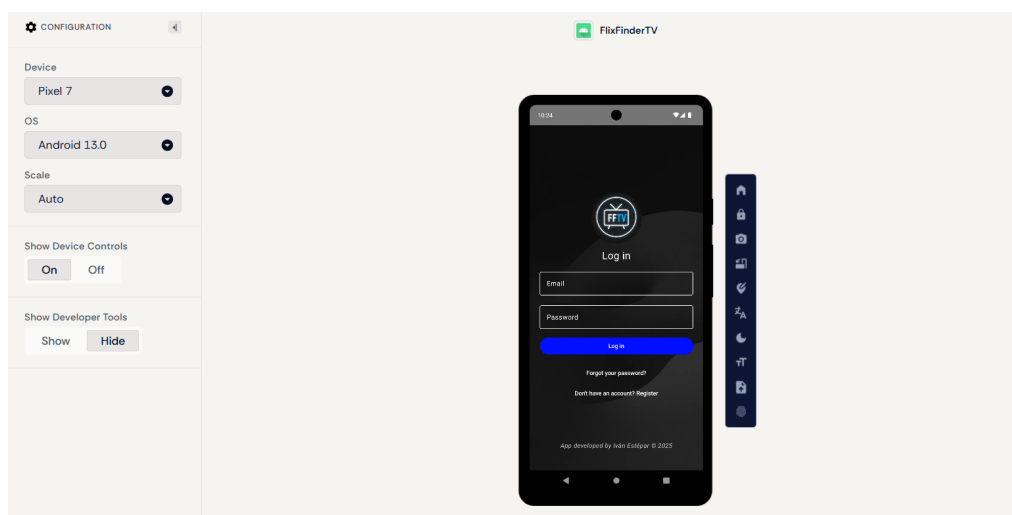


Figura 5.7: Emulador de *Appetize*

Cabe señalar que, debido a la utilización del *plan gratuito* de *Appetize*, existe una limitación de tiempo por sesión: cada emulación tiene una duración máxima de **3 minutos**. Además, el uso total de la plataforma está restringido a un máximo de **60 minutos**. Una vez transcurrido este periodo, la instancia se detiene automáticamente y la aplicación se reinstala, no manteniéndose el guardado de sesión y permitiendo continuar con la demostración desde el inicio de la aplicación. Aunque esta restricción puede requerir repetir la carga de la *app* para explorar todas sus funcionalidades, no impide evaluar correctamente la operatividad general del sistema.

Es importante tener en cuenta que, al tratarse de un emulador que se ejecuta en la nube, el rendimiento puede ser ligeramente inferior al de un dispositivo físico. Esto se traduce en tiempos de carga algo mayores y en una respuesta menos fluida durante ciertas interacciones, especialmente en aplicaciones con interfaces visuales complejas, como es el caso de la mía. No obstante, sigue siendo una herramienta muy útil para la revisión y validación del funcionamiento general de la aplicación.

La aplicación se puede probar directamente a través del siguiente enlace:

https://appetize.io/app/b_r5punogm6g4kzyxya66d2zrjji

5.7. Testing

Durante el desarrollo del proyecto se llevaron a cabo pruebas unitarias para comprobar el correcto funcionamiento de la lógica de negocio. No se incluyeron pruebas de interfaz de usuario, concentrando los esfuerzos en las capas de negocio y datos.

Pruebas Unitarias

Se crearon pruebas unitarias para los *ViewModels*, responsables de gestionar la lógica de las pantallas, asegurando que las funciones encargadas de recuperar y procesar los datos desde la base de datos local (*Room*) y la nube (*Firestore*) estuvieran correctamente implementadas. Además, se desarrollaron pruebas específicas para las funcionalidades de inicio de sesión, registro y recuperación de contraseña, garantizando así la fiabilidad de estos procesos críticos de autenticación y gestión de usuarios.

Se utilizó la librería *JUnit4* para las pruebas unitarias y *MockK* para simular el comportamiento de las dependencias externas, como la API de *TMDB* y *Firestore*.

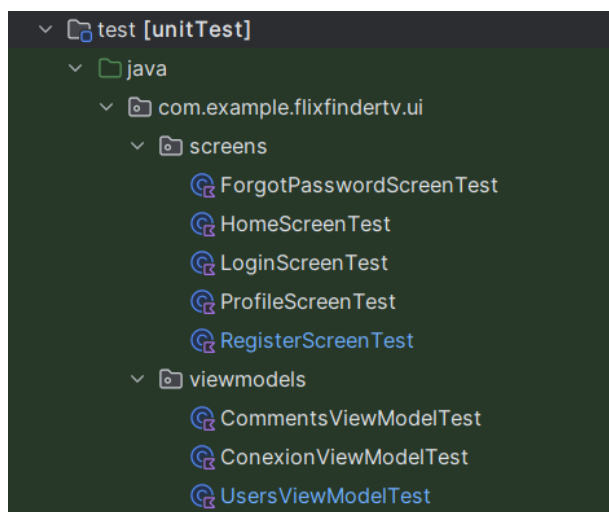


Figura 5.8: Organización de carpetas en el *testing*

Para una mejor comprensión, las funcionalidades evaluadas se agrupan en dos categorías principales: aquellas relacionadas con la lógica y reglas internas de la aplicación, y aquellas que involucran la interacción con servicios externos.

Funcionalidades de negocio

- **Perfil de usuario:** Se realizaron pruebas sobre funcionalidades relacionadas con el perfil, incluyendo el cierre de sesión y la obtención de las películas y series marcadas como favoritas por el usuario. Se verificó que estas operaciones se ejecutaran correctamente y reflejaran los datos almacenados en *Firestore*.
- **Manejo de la conectividad:** Se comprobaron los comportamientos esperados de la aplicación en situaciones de conectividad activa o intermitente, evaluando cómo responde la lógica de obtención de datos ante estos cambios.
- **Gestión de comentarios y respuestas:** Se comprobó que al escribir un comentario en una película o serie, este se almacena correctamente en la colección `comentarios` de *Firestore*. Además, se verificó que las respuestas a comentarios existentes se guardan adecuadamente, manteniendo la estructura jerárquica esperada y permitiendo una correcta recuperación y visualización en la aplicación.

Interacción con servicios externos

- **Autenticación de usuarios:** Se probaron los procesos de inicio de sesión, registro y recuperación de contraseña utilizando *Firebase Authentication*, verificando que las credenciales fueran gestionadas correctamente y que los flujos de autenticación funcionaran como se esperaba.
- **Obtención de contenido desde *Firestore*:** Se evaluó la correcta recuperación de películas y series almacenadas en la nube, específicamente la carga de contenido según el género principal del usuario. Las pruebas confirmaron que los datos se obtienen correctamente desde la colección correspondiente en *Firestore* y *Room*.

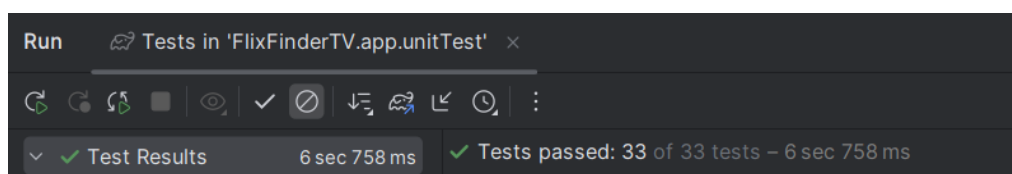


Figura 5.9: Resultados de los *tests*

5.8. Métricas de calidad

Se han implementado tests que cubren una gran parte de la funcionalidad; sin embargo, las métricas de cobertura calculadas por *SonarCloud* no alcanzan los niveles esperados. Esto se debe principalmente a que, durante el desarrollo del proyecto, el foco principal estuvo en implementar correctamente la lógica de negocio, la integración con APIs externas y la interfaz de usuario, mientras que la escritura de pruebas unitarias quedó como una tarea secundaria debido a limitaciones de tiempo.

No obstante, se han implementado un total de **33 tests unitarios**, enfocados principalmente en la validación de ***ViewModels*** y funciones auxiliares clave dentro de la lógica de negocio. Estas pruebas fueron útiles para garantizar el correcto funcionamiento de partes esenciales del sistema.

Además, dado que se trata de una aplicación Android basada en ***Jetpack Compose***, ***ViewModels***, ***Room*** y otros componentes acoplados al ciclo de vida del sistema, escribir pruebas efectivas requiere la configuración de entornos de prueba específicos y el uso de herramientas adicionales como ***MockK***, ***Robolectric*** o ***Hilt para testing***, lo que implica una inversión considerable de tiempo que no fue viable durante el desarrollo activo.

Por otro lado, se decidió no incluir pruebas de *UI* automatizadas, ya que el enfoque del proyecto estuvo centrado en las funcionalidades clave y en la experiencia de usuario. Esta ausencia también repercutió en el bajo porcentaje de cobertura, ya que muchas rutas del código relacionadas con la interacción del usuario no se ejecutan durante los análisis, por lo que no se contabilizan como cubiertas.

5.9. Créditos a librerías y contribuciones externas

Una de las funcionalidades clave de la aplicación es la integración de un reproductor de *YouTube* para la visualización de trailers relacionados con las películas y series recomendadas. Para implementar esta funcionalidad, se utilizó la librería *Android YouTube Player*, proporcionada por Pierfrancesco Soffritti [25]. Esta librería facilitó la integración del reproductor de *YouTube* dentro de la aplicación, permitiendo una experiencia de usuario fluida y sin interrupciones. Agradezco su excelente trabajo y documentación, que simplificaron significativamente el proceso de integración.

Asimismo, la aplicación se beneficia del acceso a una gran base de datos de películas y series gracias a la **API de The Movie Database**. Quiero expresar mi agradecimiento a los desarrolladores y mantenedores de *TMDB* por proporcionar esta herramienta gratuita y accesible, que ha sido fundamental para el correcto funcionamiento y enriquecimiento del contenido de la aplicación. Su esfuerzo y dedicación han sido esenciales para que el proyecto pueda ofrecer información actualizada y completa a los usuarios.

6. Trabajos relacionados

6.1. Rotten Tomatoes



Figura 6.1: Logo de Rotten Tomatoes

Rotten Tomatoes es una de las plataformas más reconocidas en el mundo del cine, y es especialmente conocida por ofrecer reseñas y calificaciones tanto de críticos como de usuarios. Aunque no utiliza algoritmos avanzados como algunos sistemas modernos de recomendación, su enfoque se basa en proporcionar sugerencias personalizadas a los usuarios a partir de las calificaciones generales y las tendencias actuales en el mercado. Esto lo convierte en una herramienta útil para quienes buscan orientación sobre qué ver, ya que presenta una variedad de películas basadas en la evaluación colectiva del público [27].

- **Enfoque:** La plataforma recomienda películas en función de las clasificaciones realizadas tanto por críticos como por usuarios comunes.
- **Impacto:** A lo largo de los años, Rotten Tomatoes ha sido una fuente confiable para millones de personas que buscan contenido de calidad, ya

sea para ver en el cine o en casa. Su sistema de calificación ha ayudado a los usuarios a tomar decisiones informadas sobre las películas que eligen, y ha establecido un estándar en la industria para evaluar la calidad del cine.

6.2. JustWatch



Figura 6.2: Logo de JustWatch

JustWatch ha ganado popularidad por su capacidad para ayudar a los usuarios a descubrir dónde pueden ver sus películas y series favoritas a través de múltiples plataformas de streaming. A pesar de no utilizar algoritmos sofisticados como los sistemas de recomendación basados en aprendizaje automático, JustWatch personaliza las sugerencias basándose en las preferencias previas de los usuarios y en las opciones de streaming disponibles. La simplicidad de la interfaz y la información precisa sobre dónde se puede encontrar un contenido en particular ha hecho de JustWatch una herramienta esencial para quienes quieren saber en qué plataformas pueden disfrutar sus programas favoritos [21].

- **Enfoque:** Ofrece recomendaciones basadas en la disponibilidad de contenido a través de diversas plataformas de streaming.
- **Impacto:** La plataforma ha facilitado el acceso a millones de usuarios, permitiéndoles encontrar fácilmente las películas y series que desean ver sin tener que buscar en cada servicio de streaming por separado. Esto ha simplificado enormemente la experiencia del usuario al ofrecer un solo lugar donde consultar la disponibilidad de contenidos.

6.3. FilmAffinity



Figura 6.3: Logo de FilmAffinity

FilmAffinity es una red social especializada en cine, que permite a los usuarios calificar y compartir sus opiniones sobre películas. La plataforma proporciona recomendaciones personalizadas en función de las calificaciones de otros usuarios que comparten gustos similares. A diferencia de otros servicios, FilmAffinity destaca por su comunidad activa y por el enfoque social en la recomendación de películas, donde las interacciones y valoraciones de los usuarios juegan un papel central en el proceso de descubrimiento [4].

- **Enfoque:** Las recomendaciones están basadas en las preferencias de los usuarios y en las valoraciones realizadas por la comunidad.
- **Impacto:** La plataforma ha logrado crear una comunidad muy activa de cinéfilos, lo que ha permitido a los usuarios descubrir nuevas películas basándose en las recomendaciones de personas con gustos similares. Esta interacción social ha hecho que FilmAffinity se convierta en una de las plataformas más valoradas por los aficionados al cine.

6.4. Trakt.tv



Figura 6.4: Logo de Trakt.tv

Trakt.tv permite a los usuarios hacer un seguimiento detallado de las películas y series que han visto, y ofrece recomendaciones personalizadas basadas en esa información. Los usuarios pueden crear listas personalizadas de contenido, lo que les permite organizar y seguir su progreso en series y películas. Trakt.tv también permite descubrir contenido nuevo basándose en las visualizaciones de amigos o personas con intereses similares [31].

- **Enfoque:** Recomendaciones basadas en el seguimiento manual de lo que los usuarios ven, junto con listas personalizadas y el intercambio de contenido con amigos.
- **Impacto:** Trakt.tv ha creado una comunidad en la que los usuarios pueden compartir y descubrir contenido nuevo basándose en los hábitos de visualización de otros, lo que fomenta el descubrimiento de películas y series que de otro modo podrían pasar desapercibidas.

6.5. Letterboxd (Versión Simplificada)

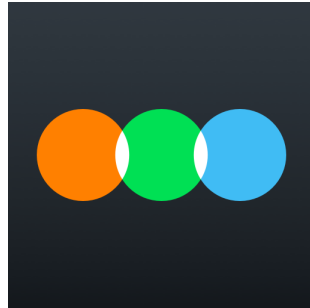


Figura 6.5: Logo de Letterboxd

Letterboxd es una plataforma que permite a los usuarios registrar las películas que han visto y compartir sus opiniones. Aunque ofrece una opción de crear listas personalizadas y compartir contenido con otros usuarios, sus recomendaciones se apoyan principalmente en las interacciones sociales y en las valoraciones de la comunidad. A través de esta plataforma, los usuarios pueden descubrir nuevas películas basándose en las preferencias y comentarios de otros, lo que fomenta una experiencia centrada en la comunidad [22].

- **Enfoque:** Las recomendaciones están basadas en la interacción social, donde las calificaciones y listas personalizadas de otros usuarios juegan un papel fundamental.
- **Impacto:** Letterboxd ha conseguido una comunidad activa de amantes del cine, proporcionando una plataforma donde los usuarios pueden descubrir y recomendar películas de manera más orgánica, sin la necesidad de complejos algoritmos de recomendación.

6.6. IMDb



Figura 6.6: Logo de IMDb

IMDb, probablemente una de las plataformas más conocidas para obtener información sobre cine, permite a los usuarios calificar y revisar películas y series. Además de mostrar recomendaciones personalizadas basadas en las calificaciones de otros usuarios, IMDb también ofrece sugerencias de títulos relacionados con las películas que el usuario ya ha visto. Aunque IMDb tiene un sistema más complejo de recomendaciones, su versión más sencilla se basa principalmente en mostrar al usuario títulos similares basándose en sus preferencias y visualizaciones anteriores [15].

- **Enfoque:** Recomendaciones basadas en la popularidad, las calificaciones de los usuarios y títulos similares a los previamente vistos.
- **Impacto:** IMDb sigue siendo una plataforma fundamental tanto para los profesionales del cine como para los aficionados, gracias a la fiabilidad de sus calificaciones y su capacidad para ofrecer recomendaciones de calidad, respaldadas por una vasta base de usuarios.

7. Conclusiones y Líneas de trabajo futuras

7.1. Conclusiones

A lo largo del desarrollo de este proyecto se ha logrado construir una aplicación de recomendación de películas y series que ofrece a los usuarios una experiencia personalizada, permitiéndoles encontrar contenido relevante basado en sus preferencias de manera sencilla y accesible. El objetivo principal de facilitar la búsqueda de contenido atractivo y adecuado para cada usuario ha sido alcanzado, proporcionando un sistema intuitivo y fácil de usar.

Entre las principales conclusiones obtenidas, destacan:

- **Estrategia de recomendaciones efectiva:** El sistema de recomendaciones basado en los gustos y preferencias de los usuarios ha demostrado ser eficiente, ofreciendo contenido relevante sin necesidad de algoritmos complejos.
- **Plataformas de almacenamiento eficientes:** *Firebase* e *Imgur* fueron las plataformas elegidas para almacenar y gestionar los datos del usuario, y su integración ha sido fácil y efectiva. *Firebase* facilitó la gestión en tiempo real de los datos, mientras que *Imgur* ofreció una solución eficiente para las imágenes de perfil, permitiendo una escalabilidad adecuada con el crecimiento de usuarios y contenido.
- **Interfaz de usuario sencilla y accesible:** La interfaz ha sido diseñada para ser simple y directa, lo que facilita la navegación y

mejora la experiencia de usuario. Además, es adaptable a diferentes dispositivos y tamaños de pantalla, optimizando la experiencia en diversos entornos.

- **Integración de *Gemini* para trivia y moderación de comentarios:** *Gemini* ha mejorado la experiencia de trivia generando preguntas dinámicas y adaptadas a los intereses de los usuarios. Además, ha sido clave en la moderación de comentarios, asegurando que no se publiquen mensajes inapropiados y promoviendo un ambiente respetuoso.
- **Desempeño de la aplicación en diferentes entornos:** La *app* ha demostrado un rendimiento consistente incluso en condiciones de conectividad limitada, gracias a su capacidad de ofrecer contenido tanto *online* como *offline*, lo que asegura una experiencia fluida sin importar la calidad de la conexión.
- **Escalabilidad y eficiencia del sistema de base de datos:** El uso de *Room* para el almacenamiento local de películas y series ha garantizado un manejo eficiente de grandes volúmenes de datos. La *app* puede almacenar hasta 2000 ítems sin afectar el rendimiento, lo que ha sido esencial para las recomendaciones personalizadas y la visualización *offline*.
- **Adaptación a diferentes preferencias de usuario:** La aplicación se ajusta dinámicamente a las preferencias individuales de cada usuario, mejorando las recomendaciones a medida que interactúan con la *app*. Este enfoque ha sido crucial para ofrecer contenido cada vez más relevante y personalizado.
- **Enfoque en la privacidad y seguridad del usuario:** Se implementaron mecanismos robustos para proteger la información personal de los usuarios, asegurando que el acceso sea seguro mediante sistemas de autenticación confiables y procesos de recuperación de contraseña protegidos.

El sistema ha sido probado y validado en dispositivos reales, demostrando su capacidad para recomendar contenido relevante basado en las preferencias de los usuarios, así como para facilitar el descubrimiento de nuevo contenido y fomentar la participación social dentro de la comunidad. El funcionamiento ha sido adecuado y eficiente. A pesar de los logros alcanzados, se ha identificado que siempre existen áreas de mejora, lo que abre diversas oportunidades para futuras ampliaciones y optimizaciones de la aplicación.

7.2. Líneas de trabajo futuras

A pesar de que el proyecto ha alcanzado sus objetivos principales, existen varias áreas en las que se puede continuar trabajando para mejorar la aplicación y expandir sus capacidades. Algunas de las líneas de trabajo futura son las siguientes:

- **Implementación de un sistema de recomendaciones basado en IA:** Integrar un algoritmo de aprendizaje automático, como un sistema de filtrado colaborativo o basado en contenido, podría mejorar significativamente la precisión de las recomendaciones, adaptándolas mejor a las preferencias individuales de los usuarios. Un sistema de recomendaciones inteligente no solo consideraría las interacciones pasadas de los usuarios, sino también patrones de comportamiento de usuarios similares, lo que permitiría ofrecer recomendaciones más precisas y atractivas.
- **Ampliación de la base de datos de películas y series:** Ampliar la integración con bases de datos de contenido, como *TMDb* o APIs de servicios de *streaming*, podría enriquecer las recomendaciones y permitir que la aplicación ofrezca una gama más amplia de títulos y géneros. Esto no solo enriquecería las opciones de entretenimiento para los usuarios, sino que también ayudaría a mantener la aplicación actualizada con las últimas novedades del mundo del cine y la televisión.
- **Optimización del rendimiento y escalabilidad:** A medida que la base de usuarios crezca, será necesario mejorar la infraestructura de la aplicación para garantizar un rendimiento eficiente y sin interrupciones. Especialmente en lo que respecta a la carga de contenido y las consultas de búsqueda, la aplicación debería ser capaz de manejar un mayor volumen de tráfico sin que la experiencia de usuario se vea afectada. Esto implicaría optimizar consultas de bases de datos, mejorar tiempos de respuesta y asegurar la estabilidad del servidor ante un aumento en el número de usuarios simultáneos.
- **Integración de más servicios de streaming:** Incorporar opciones que permitan conectar directamente con las plataformas de streaming de video, de modo que la aplicación pueda consultar sus catálogos oficiales mediante APIs y así mostrar no solo recomendaciones, sino también la disponibilidad exacta de cada contenido en dichas plataformas (por ejemplo, Netflix, Disney+, HBO). Esto brindaría al usuario información precisa sobre dónde puede ver la película o serie

recomendada. Además, sería útil implementar un sistema automático que detecte si el tráiler disponible, ya sea alojado en YouTube u otra plataforma, puede reproducirse correctamente; en caso de detectar problemas (como enlaces rotos o videos inaccesibles), el sistema evitaría mostrar ese tráiler para mejorar la experiencia del usuario.

- **Continuar las estrategias de gamificación y ampliar la funcionalidad del trivia:** La inclusión de modos de juego adicionales, como competiciones de trivia entre usuarios o desafíos personalizados, podría agregar una dimensión social y divertida a la experiencia de usuario. Estos modos de trivia podrían incluir *rankings*, recompensas y desafíos, haciendo que la aplicación no solo sea una herramienta de recomendación de contenido, sino también un lugar para el entretenimiento interactivo.
- **Soporte multiplataforma:** Actualmente, la aplicación está orientada a dispositivos *Android*. Una línea de trabajo futura podría ser expandir la aplicación a otras plataformas, como *iOS*, lo que abriría la aplicación a un público más amplio. Además, la integración con plataformas web podría ofrecer una experiencia más flexible para los usuarios que deseen acceder al contenido desde diferentes dispositivos.
- **Ampliación del conjunto de pruebas:** Para garantizar la fiabilidad y robustez de la aplicación, sería conveniente ampliar la cobertura de pruebas unitarias en los distintos componentes del sistema, especialmente en las funcionalidades críticas como la generación de recomendaciones y el guardado *offline*. Además, implementar pruebas de integración podría ayudar a detectar errores en la interacción entre módulos y asegurar un comportamiento coherente de la aplicación en distintos escenarios.

Bibliografía

- [1] Appetize.io. Appetize.io: Run mobile apps in your browser. <https://appetize.io>, 2025.
- [2] The Cryptography Developers. Fernet: Symmetric encryption system. <https://cryptography.io/en/latest/fernet/>, 2025.
- [3] Figma. Figma – interface design tool. <https://www.figma.com>, 2025.
- [4] FilmAffinity. FilmAffinity – tu página personalizada de votación y recomendaciones de cine. <https://www.filmaffinity.com>, 2003.
- [5] Python Software Foundation. Python language reference, version 3.x. <https://www.python.org>, 2025.
- [6] GitHub. GitHub – where the world builds software. <https://github.com>, 2008.
- [7] GitHub. GitHub Actions – automation for software workflows. <https://github.com/features/actions>, 2019.
- [8] Google. Android Studio – official ide for android development. <https://developer.android.com/studio>, 2013.
- [9] Google. Firebase Firestore – serverless nosql database. <https://firebase.google.com/docs/firestore>, 2017.
- [10] Google. ViewModel – a component for managing ui-related data. <https://developer.android.com/topic/libraries/architecture/viewmodel>, 2018.

- [11] Google. Jetpack Compose – a modern toolkit for building native ui. <https://developer.android.com/jetpack/compose>, 2020.
- [12] Google. Gemini – ai-powered language understanding system. <https://gemini.ai>, 2023.
- [13] Google. Room persistence library. <https://developer.android.com/training/data-storage/room>, 2025.
- [14] Gradle. Gradle Kotlin DSL – kotlin-based domain-specific language for gradle. https://docs.gradle.org/current/userguide/kotlin_dsl.html, 2018.
- [15] IMDb. IMDb – movies, tv and celebrities. <https://www.imdb.com>, 1990.
- [16] IMDb. IMDb API – movie and tv information. <https://www.imdb.com/interfaces>, 2002.
- [17] Imgur. Imgur – image hosting and sharing. <https://imgur.com>, 2009.
- [18] JetBrains. Kotlin – modern programming language for jvm. <https://kotlinlang.org>, 2011.
- [19] JetBrains. Kotlin coroutines overview. <https://kotlinlang.org/docs/coroutines-overview.html>, 2025.
- [20] Jigsaw. Perspective API – analyze text for toxicity. <https://www.perspectiveapi.com>, 2017.
- [21] JustWatch. JustWatch – discover where to watch movies and tv shows. <https://www.justwatch.com>, 2014.
- [22] Letterboxd. Letterboxd – your life in film. <https://letterboxd.com>, 2011.
- [23] OpenAI. ChatGPT – conversational ai. <https://openai.com/chatgpt>, 2022.
- [24] Overleaf. Overleaf – online latex editor. <https://www.overleaf.com>, 2025.
- [25] Pierfrancesco Soffritti. Android YouTube Player – a youtube player for android. <https://github.com/PierfrancescoSoffritti/Android-YouTube-Player>, 2020.

- [26] Red Hat. ¿Qué es una API REST? <https://www.redhat.com/es/topics/api/what-is-a-rest-api>, 2025.
- [27] Rotten Tomatoes. Rotten Tomatoes – movie reviews and ratings. <https://www.rottentomatoes.com>, 1998.
- [28] Scrum Guide. The Scrum Guide. <https://www.scrumguides.org>, 2025.
- [29] SonarSource. SonarCloud – cloud-based code quality and security. <https://sonarcloud.io>, 2016.
- [30] The Movie Database. TMDb – the movie database api. <https://www.themoviedb.org>, 2008.
- [31] Trakt.tv. Trakt.tv – automatically track what you’re watching. <https://trakt.tv>, 2011.
- [32] Wikipedia. Jean-Luc Godard – wikipedia, the free encyclopedia. https://en.wikipedia.org/wiki/Jean-Luc_Godard, 2025.
- [33] Wikipedia. NoSQL – wikipedia, the free encyclopedia. <https://es.wikipedia.org/wiki/NoSQL>, 2025.